

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
**«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ»**
(Н И У « Б е л Г У »)

ИНСТИТУТ ИНЖЕНЕРНЫХ И ЦИФРОВЫХ ТЕХНОЛОГИЙ

Кафедра информационных и робототехнических систем

**РАЗРАБОТКА ПРОГРАММЫ "КОМПЬЮТЕРНЫЙ МАГАЗИН" С
ИСПОЛЬЗОВАНИЕМ ОБЪЕКТОВ, СОДЕРЖАЩИХ ИНФОРМАЦИЮ
О ДЕЯТЕЛЬНОСТИ КОМПЬЮТЕРНОГО МАГАЗИНА**

Курсовая работа
по дисциплине «Объектно-ориентированное программирование»
студента очной формы обучения
направления подготовки 09.03.02 Информационные системы и технологии
(код, наименование направления подготовки)
2 курса группы 12002408
Солодова Егора Николаевича

Научный руководитель:

ассистент

(степень, должность)

Пузырев Никита Сергеевич

(Ф.И.О. руководителя)

БЕЛГОРОД 2026

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ	2
ВВЕДЕНИЕ	3
1. ТЕОРЕТИЧЕСКАЯ ЧАСТЬ	5
1.1 Общая характеристика деятельности Компьютерного магазина.....	5
1.2 Определение функциональных требований к разработке	9
1.3 Обоснование выбора языка программирования и среды разработки.....	11
2. ПРАКТИЧЕСКАЯ ЧАСТЬ	12
2.1 Разработка UML-диаграммы классов	12
2.2 Реализация программы	24
2.3 Демонстрация работы программы.....	55
ЗАКЛЮЧЕНИЕ	80
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	82
ПРИЛОЖЕНИЕ А	84

ВВЕДЕНИЕ

Динамическое развитие информационных технологий, повышение доступности и качества компьютерных устройств привели к повседневному их использованию в жизни человека для решения как рутинных, так и профессиональных задач. В связи с чем в России, как и во всем мире, сформировался устойчивый и конкурентный рынок компьютерной техники [3, 4].

Чтобы не проиграть борьбу за внимание и доверие клиентов на данном рынке, каждая компания стремится улучшать сервис, качество предоставляемых услуг, расширить ассортимент поставщиков и товаров. Кроме того, из-за обилия и разнообразия компьютерной техники клиент в момент покупки сталкивается с неминуемым и сложным выбором. Чтобы помочь покупателю, компании обязаны облегчить ему процесс поиска, подбора и покупки соответствующего устройства [8, 10].

В связи с чем повсеместной, распространённой и обязательной практикой в данном бизнесе является внедрение современных средств автоматизации ключевых процессов работы компьютерного магазина, призванное решить ключевые проблемы данных компаний [3]. Поэтому тема работы актуальна.

Цель курсовой работы: разработать консольное приложение «Компьютерный магазин» на языке программирования Java, содержащее информацию о деятельности компьютерного магазина.

Для достижения поставленной цели необходимо решить следующие задачи:

изучить необходимый теоретический материал и провести анализ предметной области «Компьютерный магазин»;

провести проектирование архитектуры программы: разработать иерархию классов, основываясь на принципах объектно-ориентированного программирования, определить взаимосвязи между ними [10];

реализовать основные функции программы: поиск, покупка, регистрация, удаление, изменение, товара, регистрация и авторизация продавцов и клиентов;

организовать хранение коллекций объектов покупателей, продавцов и товаров;

реализовать удобный консольный интерфейс программы.

Объект исследования курсовой работы: компьютерный магазин, предоставляющий услуги поиска и покупки электронных устройств.

Предмет исследования курсовой работы: алгоритм, моделирующий деятельность компьютерного магазина.

Курсовая работа включает 2 раздела.

В первом разделе рассматривается предметная область «Компьютерный магазин», выделяются функциональные требования к разработке программы.

Во втором разделе производится разработка UML-диаграммы классов, реализация программы на языке программирования Java в виде консольного приложения, а также демонстрация её работоспособности.

Курсовая работа состоит из 83 страниц, 62 иллюстраций и 1 приложения.

1. ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

1.1 Общая характеристика деятельности Компьютерного магазина

Компьютерный магазин является специализированным предприятием розничной торговли, основным видом деятельности которого является продажа своего ассортимента товаров, связанных с информационными технологиями. К ним относятся готовые персональные компьютеры и ноутбуки, комплектующие: процессоры, видеокарты, оперативная память, накопители; периферийные устройства: мониторы, клавиатуры, мыши; а также сетевое оборудование и программное обеспечение [10]. Главным источником дохода является продажа этих товаров.

До широкого внедрения информационных технологий в повседневную жизнь человека и массового распространения автоматизации и цифровизации рыночного взаимодействия между продавцом и покупателем существовал только традиционный процесс обслуживания клиентов [3].

Традиционный процесс обслуживания клиента в компьютерном магазине представляет собой следующую последовательность взаимодействий между покупателем и персоналом магазина:

- 1) первичная консультация клиента. Клиент посещает компьютерный магазин. Его приветствует и встречает консультант-продавец, задача которого — узнать запрос покупателя. При позитивном стечении обстоятельств завязывается диалог, в котором уточняются ключевые параметры: цель использования устройства: офисная работа, домашнее пользование, узкоспециализированные задачи; бюджет, предпочтения по брендам или конкретным техническим характеристикам [4];

- 2) подбор и демонстрация товара. На основе полученных от клиента данных продавец-консультант предлагает несколько подходящих вариантов из имеющегося ассортимента. Для таких товаров, как ноутбуки или мониторы,

зачастую возможна моментальная демонстрация: включение, проверка изображения, качества продукта. В случае сборки персонального компьютера на заказ, клиенту предоставляется возможность выбрать каждую комплектующую отдельно, основываясь как на его вышеперечисленных потребностях, так и на их совместимости [8];

3) оформление покупки. После покупатель выражает свой конечный вердикт, то есть согласен ли тот на покупку или нет. В случае одобрения продавец переходит к стандартной процедуре оформления товара [10]:

I. формирование документа, подтверждающего покупку, чаще всего чека, в котором указываются: название компьютерного магазина, в котором покупается товар, его адрес, сумма, дата покупки, ФИО и должность продавца-консультанта и покупателя, участвующих в процессе купли-продажи;

II. передача покупателю полного комплекта: товара в оригинальной упаковке, гарантийного талона, технической документации;

III. оплата наличными средствами через кассовый аппарат или по банковской карте с использованием терминала.

4) дальнейшее обслуживание. После совершения покупки клиент сохраняет право на гарантийное и сервисное обслуживание. При выявлении недостатков товар может быть возвращен или обменян в установленном законом порядке.

Важной операцией, скрытой от клиента, является логистика и управление запасами. Ответственные сотрудники ведут не прекращающийся в течении всей смены учет остатков товара на складе, проводят плановые инвентаризации и формируют заявки поставщикам на покупку новых продуктов для пополнения ассортимента, что является основой для бесперебойной работы компьютерного магазина [3, 4].

Такой процесс имеет некоторые выявленные недостатки, связанные в основном с взаимодействием покупателя и консультанта [8, 10]:

- не всем покупателям нужна консультация, например если клиент достаточно хорошо разбирается в той или иной цифровой технике;
- многие клиенты предвзято считают, что основной задачей консультантов является навязывание им покупки неликвидного товара, что отчасти является верным суждением;
- покупатели охотнее верят отзывам знакомых о том или ином товаре, нежели консультанту;
- некоторые покупатели достаточно застенчивы и могут из-за этого отказаться от помощи консультанта, а потом жалеть о неудачно купленном товаре и возможно винить в этом себя, но зачастую магазин или даже продавцов-консультантов;
- продавцы-консультанты в свою очередь мотивированы продать как можно больше единиц продуктов в день, особенно тех, что не пользуются популярностью у клиентов, на как на одном из факторов основывается их заработная плата, из-за чего падает качество обслуживания каждого из множества клиентов.

Однако с развитием цифровых технологий и интернет-инфраструктуры в России крупнейшие игроки рынка, такие как «DNS», изначально развивавшийся как классическая сеть розничных магазинов, сделали ставку на развитие интернет-ресурса, благодаря которому покупатель, если того желает, лишает себя продавца-консультанта как одного из звеньев процесса подбора и покупки товара, и может использовать в качестве его замены все возможности и преимущества интернет-магазина. [3, 8].

Преимущества онлайн-компьютерного магазина для клиента кардинально меняют потребительский опыт [8]:

- быстрый и точный поиск и навигация. В отличие от ограниченного пространства физической витрины и знаний продавца-консультанта сайт предоставляет доступ ко всему каталогу, который может насчитывать десятки тысяч позиций. Клиент может мгновенно найти нужную модель по названию или описанию;

- расширенная фильтрация и сортировка. Это ключевое конкурентное преимущество. Покупатель может отобрать товары по десяткам параметров: цене, производителю, конкретным техническим характеристикам, рейтингу, наличию. Это превращает процесс подбора из диалога с консультантом в самостоятельный, осознанный и эффективный поиск;

- сравнение. Онлайн-магазины позволяют выделить несколько товаров и наглядно сравнить их характеристики в единой таблице, что практически невозможно сделать в офлайн-магазине без значительных временных затрат;

- корзина и отложенные покупки. Виртуальная корзина позволяет клиенту формировать список будущих покупок, редактировать его, откладывать товары на будущее и видеть итоговую сумму до момента оплаты.

Таким образом, современный компьютерный магазин существует в двух взаимодополняющих форматах: физическом, обеспечивающем живой контакт, демонстрацию и немедленное получение товара, и онлайн, предоставляющем неограниченный выбор, удобные инструменты анализа и максимальную доступность. Разрабатываемая в рамках данной работы программа моделирует ключевые бизнес-процессы, общие для обеих моделей: управление товарным каталогом, обработку заказов и финансовый учет.

1.2 Определение функциональных требований к разработке

Для формирования функциональных требований к программе «Компьютерный магазин» необходимо определить ключевые роли пользователей, которые будут с ней взаимодействовать, а также их возможности:

- продавец. Пользователь с максимальным уровнем доступа и контроля, может формировать и редактировать каталог товаров, и количество продуктов на складе;
- клиент. Основной пользователь сервиса. Имеет возможность просматривать каталог, использовать инструменты поиска и фильтрации, формировать заказы и свою корзину, совершать покупки, просматривать свой список покупок;
- товары. Набор сущностей, которые будут хранить в себе характеристики тех или иных продуктов, предоставленных в каталоге [1].

Исходя из выделенных ролей и анализа предметной области, были сформулированы следующие функциональные требования к программе:

- программа должна обеспечивать возможность создания, редактирования и удаления позиций товаров в каталоге;
- каждый товар должен быть представлен в виде объекта соответствующего класса;
- для каждого класса товара должны быть предусмотрены специфические поля для ввода характеристик и методы для их корректного заполнения [1];
- при совершении клиентом покупки товар автоматически должен попадать в его список покупок и списываться со склада;

- в программе должна быть предусмотрена возможность покупателю добавлять понравившиеся товары в корзину;
- в программе должен быть предусмотрен механизм, не позволяющий купить товар, который отсутствует на складе как из корзины клиента, так и из основного каталога, а также в случае недостаточного количества средств на балансе покупателя;
- в программе должна быть реализована возможность просмотра всего каталога в структурированном виде, с возможностью применения фильтров и сортировок [7, 9];
- в программе должен быть реализован поиск товаров [9];
- программа должна в реальном времени отображать актуальное количество каждого товара на складе;
- в программе должны быть предусмотрены возможности для авторизации и регистрации клиентов и продавцов [7, 9];
- программа должна обладать понятным и удобным консольным интерфейсом с выводом в него каталога товаров, баланса клиента, содержимого его списка покупок и корзины [7];
- в программе должна быть реализована возможность долговременного хранения данных, то есть представления их в виде коллекций с последующей записью в соответствующие файлы [9].

Данный набор требований служит непосредственным основанием для проектирования иерархии классов, определения их полей, методов и взаимосвязей, что будет реализовано в следующем разделе работы.

1.3 Обоснование выбора языка программирования и среды разработки

Для реализации проекта «Компьютерный магазин» был выбран язык программирования Java и интегрированная среда разработки (IDE) IntelliJ IDEA Community Edition.

Обоснование выбора Java:

- строгая объектная ориентированность. Этот фактор является ключевым для данной работы, так как основная задача — представить компьютерный магазин в виде системы взаимодействующих объектов. Java позволяет использовать при написании кода все основные принципы ООП, такие как инкапсуляция, полиморфизм, абстракция и наследование [7,9];
- богатая стандартная библиотека (Java SE). Для организации хранения данных клиентов, продавцов и товаров весьма удобно использовать готовый, оптимизированный Collections Framework, предоставляющий разработчику готовый программный модуль для работы со всеми стандартными коллекциями, такими как «List», «Map», «Stack» и прочими [7,9].

Обоснование выбора IntelliJ IDEA Community Edition:

- встроенный отладчик. В процессе написания программы разработчик непременно столкнётся с появлением ошибок. Данная среда разработки обладает широким набором инструментов для выявления ошибок в написанном коде, их классификации, что помогает программисту быстрее найти подходящий путь решения возникшей проблемы [2,6];
- доступность. Community Edition разработана для бесплатного пользования и содержит весь необходимый функционал для учебного проекта.

Таким образом выбранные язык программирования и среда разработки идеально подходят для реализации поставленных задач и написания кода программы «Компьютерный магазин».

2. ПРАКТИЧЕСКАЯ ЧАСТЬ

2.1 Разработка UML-диаграммы классов

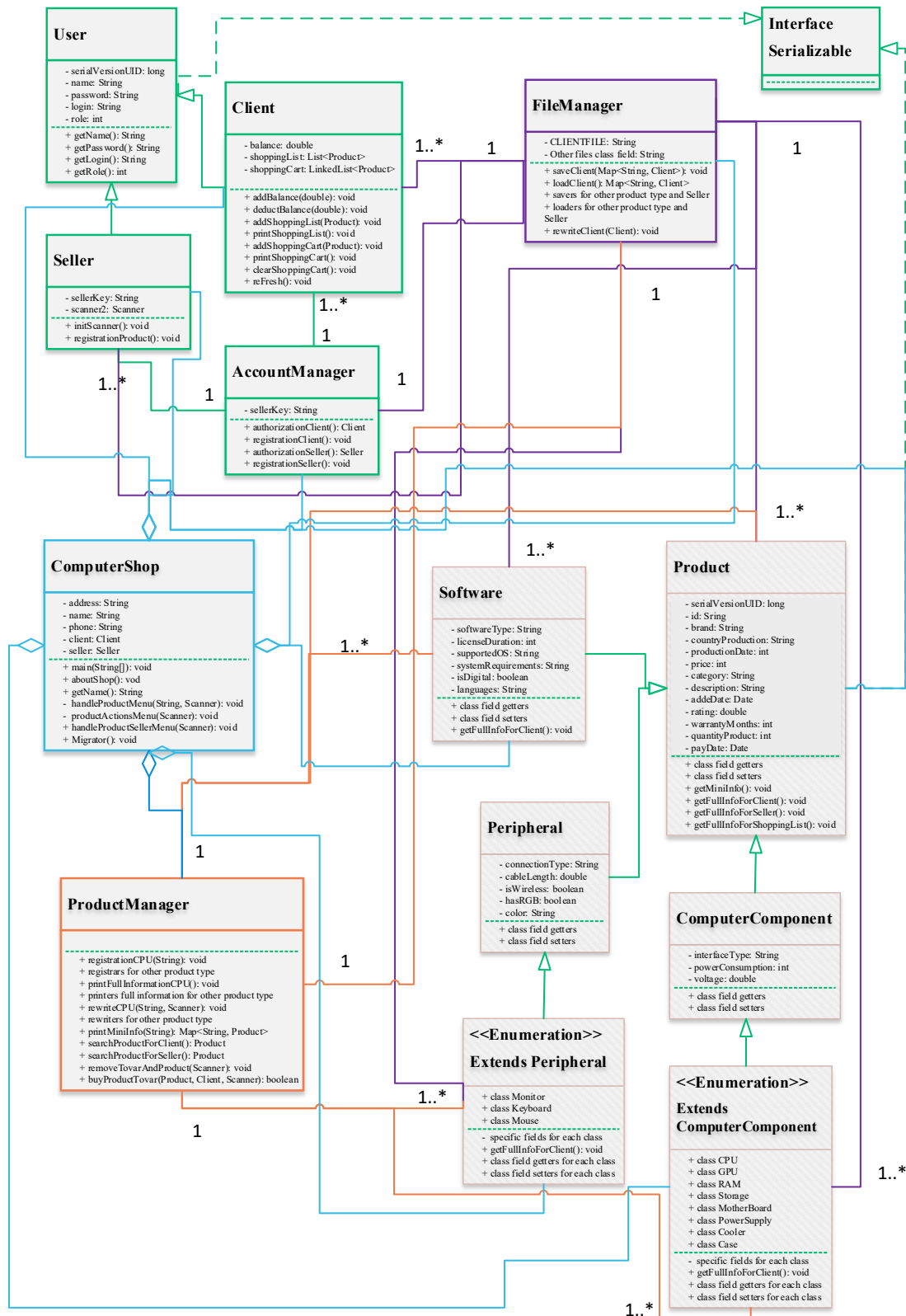


Рисунок 1 – UML-диаграмма классов «Компьютерный магазин»

Проектирование архитектуры является обязательным этапом в разработке программного обеспечения, позволяющим систематизировать структуру будущей системы. Перед тем, как начать разрабатывать программу и реализовывать поставленные задачи, следует составить модель иерархии классов и их взаимодействий. Сделать это можно многими способами, но самый удобный, точный и эффективный – разработать UML-диаграмму классов (*Unified Modeling Language* – унифицированный язык моделирования) [5]. На ней можно подробно указать, какие классы следует реализовать в программе, какие у них будут методы и поля, а также как эти классы будут взаимодействовать друг с другом. Разработанная UML-диаграмма классов для программы «Компьютерный магазин» представлена на рисунке 1.

Класс `ComputerShop` представляет собой компьютерный магазин, координирует работу экземпляров всех планируемых к разработке классов, имеет поля, хранящие в себе адрес, контактный телефон, название торговой точки, а также действующего продавца и покупателя. Данный класс имеет метод `main`, то есть является точкой входа для всей будущей программы, именно в нём будет разрабатываться консольный интерфейс, имеет метод `aboutShop` и `getName`, передающие общую информацию о магазине, а также специализированные методы для упрощения построения меню программы (`handleProductMenu` и `handleProductSellerMenu`) и работы с каталогом товаров(`productActionsMenu`).

Класс `User` содержит общие поля для пользователей компьютерного магазина, такие как имя, логин, пароль и роль. Кроме того, данный класс реализует интерфейс «`Serializable`», методы которого направлены на осуществление процесса «Сериализации», другими словами, дают возможность корректно хранить экземпляры классов в соответствующих файлах, ориентируясь на их версии. Для обозначения версии класса в `User` имеется поле `SerialVersionUID`. При помощи методов-«геттеров»: `getName`,

getLogin, getPassword и getRole; класс может свободно передавать значение этих полей.

Клас Seller является наследником User-a, то есть связан с ним в отношении наследования, другими словами – обобщения. Он представляет собой продавца-консультанта компьютерного магазина, методы initScanner и registrationProduct предназначены для добавления новых товаров в каталог доступных, поля sellerKey и scanner2 необходимы для прохождения процесса регистрации. Кроме того, класс Seller связан агрегативно с классом ComputerShop, ассоциативно с AccountManager и FileManager.

Ещё одним наследником класса User, связанным с ним в отношении наследования, является класс Client. Он представляет собой покупателя в компьютерном магазине. Для осуществления своей задачи (покупки товаров) данный класс имеет поля balance, shoppingList и shoppingCart, представляющие собой соответственно баланс (количество платёжных средств) клиента, его список и корзину покупок. Для управления балансом покупателя к разработке предусматриваются методы addBalance, deductBalance, предназначенные соответственно для его пополнения и списания с него денежных средств в случае покупки. Для управления состоянием списка купленных товаров (истории покупок) клиента следует реализовать методы addShoppingList и printShoppingList, предназначенные соответственно для добавления в него купленных товаров и вывода его содержимого в консоль. С той же целью, но для корзины клиента, следует реализовать методы addShoppingCart, printShoppingCart и clearShoppingCart, где последний должен будет в случае возникновения необходимости очищать содержимое корзины. Немаловажным я является метод refresh, планируется, что именно он будет отвечать, за актуализацию состояний товаров в корзине клиента, то есть за обновление товаров в случае их изменения в каталоге. Как и Seller, класс Client связан агрегативно с классом ComputerShop, ассоциативно с AccountManager и FileManager.

Класс AccountManager представляет собой менеджер аккаунтов пользователей программы и полностью оправдывает своё название. У него планируется реализовать две важнейшие функции: регистрацию и авторизацию продавцов и покупателей. Ответственные за это четыре метода с соответствующими названиями: authorizationClient, registrationClient, authorizationSeller, registrationSeller. Планируется в конечной версии программы сделать дополнительное условие для регистрации продавца – кодовое слово, в связи с чем данный класс имеет поле sellerKey. AccountManager агрегативно связан с ComputerShop и ассоциативно с Client, Seller, FileManager.

Класс Product представляет собой товар (далее может обозначаться словом «продукт» во избежание тавтологии), который будет помещаться в каталог продуктов и учувствовать в процессе купли-продажи. Хотя у него и будет иметься наследники, представляющие 12 конкретных категорий товаров (о них будет повествоваться далее), данный класс не является абстрактным, его экземпляры будут создаваться при каждой регистрации какого-либо нового продукта. Такой принцип выбран для того, чтобы в будущем обеспечить покупателю при выводе каталога товаров видимость только базовых их характеристик, и в случае появления интереса к какому-либо из продуктов тот уже мог получить полную информацию о товаре. Другими словами, каждый зарегистрированный товар хранится двумя экземплярами: как продукт и как, например, видеокарта. Для выполнения такой задачи планируется реализовать в классе Product следующие поля: id, brand, countryProduction, productionDate, price, category, description, addeDate, rating, warrantyMonths, quantityProduct, payDate; хранящие в себе соответственно: идентификатор, бренд, страну, дату производства, цену, категорию, описание, дату регистрации, рейтинг, срок гарантии в месяцах, количество единиц на складе, дату покупки товара. Все эти поля отобраны так чтобы сформировать минимальный допустимый набор информации, характерный для любой

категории товаров. Product с той же целью, что и User, реализует интерфейс Serializable, в следствие чего должен иметь специальное поле SerialVersionUID, хранящее в себе название версии реализации класса. Следует также реализовать методы для заполнения полей данного класса и получения информации из них, то есть набор «сеттеров» и «геттеров», методы для формирования и вывода в консольный интерфейс информации о товаре: getMiniInfo, getFullInfoForClient, getFullInfoForSeller, getFullInfoForShoppingList, соответственно для краткой информации о товаре, полной информации о товаре для клиента, продавца и для покупателя, решившего заглянуть в свой список купленных товаров. Класс Product агрегативно связан с ComputerShop, ассоциативно с ProductManager и FileManager.

Класс Software является непосредственным наследником Product, представляет собой программное обеспечение, которое будет продаваться в компьютерном магазине. Предполагаемые поля: softwareType, licenseDuration, supportedOS, systemRequirements, isDigital, languages; хранят в себе информацию соответственно о: типе программного обеспечения (операционная система, игра, офисное приложение и т.д.), сроке предоставляемой лицензии, поддерживаемых операционных системах, системных требованиях, способе выпуска и продажи (цифровое или выпускаемое на физическом носителе), поддерживаемых языках программного обеспечения. Для заполнения данных полей и передачи их значений планируется разработать для каждого из них соответствующий «сеттер» и «геттер», кроме того, следует переопределить метод getFullInfoForClient для вывода полной информации конкретно для программного обеспечения. Software агрегативно связан с ComputerShop, ассоциативно с FileManager и ProductManager.

Класс Peripheral является наследником Product и представляет собой периферийное устройство, которое будет продаваться в компьютерном

магазине. Исходя из принципа объектно-ориентированного подхода и здравой логики, объект такого класса не может существовать, так как существует множество периферийных устройств, у каждого из которых будут свои уникальные характеристики. В связи с чем, данный класс выступает в роли посредника между Product и конкретной реализацией внешнего устройства, например, клавиатурой, имеет только те поля, которые отражают общие характеристики периферийных устройств: `connectionType`, `cableLength`, `isWireless`, `hasRGB`, `color`; хранящие в себе соответственно информацию о: типе подключения, длине кабеля, принадлежности к подтипу проводных или беспроводных, наличии подсветки, цвете устройства. Для заполнения данных полей и передачи их значений планируется разработать для каждого из них соответствующий «сеттер» и «геттер».

Класс `ComputerComponent`, так же, как и `Peripheral` является наследником `Product` и представляет собой компонент компьютера, который будет продаваться в компьютерном магазине, выступает в качестве посредника между `Product` и конкретной реализацией компьютерного устройства, например, процессором или видеокартой. Имеет только те поля, которые отражают общие характеристики компьютерных компонентов: `interfaceType`, `powerConsumption`, `voltage`; хранящие в себе соответственно информацию о: типе интерфейса или собственного подключения, или подключения дополнительных внешних устройств, потребляемой мощности, рабочего напряжения устройства. Для заполнения данных полей и передачи их значений планируется разработать для каждого из них соответствующий «сеттер» и «геттер».

Enumeration «Extends `Peripheral`» представляет собой перечисление классов наследников `Peripheral`, другими словами, классов представляющих собой конкретные периферийные устройства, которые будут находиться в каталоге товаров компьютерного магазина, с специфическими для них полями:

- Class Monitor – компьютерный монитор, его специфические поля:
 - screenSize – диагональ экрана в дюймах;
 - resolution – разрешение монитора;
 - refreshRate – частота обновления экрана;
 - panelType – тип матрицы монитора;
 - responseTime – время отклика экрана;
 - hasHDR – наличие или отсутствие поддержки технологии HDR;
 - hasBuiltInSpeakers – наличие или отсутствие встроенных динамиков.
- Class Keyboard – клавиатура, специфические поля:
 - keyboardType – тип клавиатуры (механическая или мембранная);
 - countKey – количество клавиш;
 - switchType – тип переключателей (только для механических клавиатур);
 - layout – нанесённые на клавиши языковые раскладки;
 - hasNumpad – наличие или отсутствие цифрового блока.
- Class Mouse – компьютерная мышь, специфические поля:
 - dpi – чувствительность мыши;
 - buttonsCount – количество кнопок на мыши;
 - sensorType – тип сенсора мыши;
 - isGaming – относится ли мышь к игровым или нет.

Каждый из перечисленных классов должен обладать набором методов для заполнения и передачи своих полей, то есть «сеттерами» и «геттерами», а также переопределённым методом `getFullInfoForClient`, для вывода полной информации о себе в консоль. Также, все эти классы агрегативно связаны с `ComputerShop`, ассоциативно с `FileManager` и `ProductManager`.

Enumeration «Extends `ComputerComponent`» представляет собой перечисление классов наследников `ComputerComponent`, другими словами, классов представляющих собой конкретные компьютерные детали, которые

будут находиться в каталоге товаров компьютерного магазина, с специфическими для них полями:

- Class CPU – процессор компьютера, его специфические поля:
 - socket – необходимый для установки сокет материнской платы;
 - coreCount – количество ядер процессора;
 - threadCount – количество потоков процессора;
 - baseFrequency – базовая частота процессора;
 - maxFrequency – максимальная частота процессора;
 - cacheSize – размер кэша процессора третьего уровня (L3);
 - memoryType – поддерживаемые типы оперативной памяти;
 - hasIntegratedGraphics – наличие или отсутствие встроенного графического ядра.
- Class MotherBoard – материнская плата, специфические поля:
 - chipset – чипсет материнской платы;
 - socket – сокет материнской платы;
 - ramSlots – количество слотов для оперативной памяти;
 - maxRamCapacity – максимальный поддерживаемый объём оперативной памяти;
 - ramType – поддерживаемые типы оперативной памяти;
 - m2Slots – количество слотов для установки m.2 накопителей;
 - sataPorts – количество слотов для установки SATA3 накопителей;
 - hasWiFi – наличие или отсутствие встроенного Wi-Fi модуля;
 - audioCodec – название встроенной аудио карты;
 - lanSpeed – пропускная способность встроенной сетевой карты.
- Class GPU – видеокарта, специфические поля:
 - vramSize – объём видеопамяти видеокарты;
 - memoryType – тип памяти видеокарты;
 - baseFrequency – базовая частота работы видеокарты;
 - boostFrequency – максимальная частота работы видеокарты;

- cudaCores – количество производительных ядер видеокарты;
- memoryBusWidth – разрядность шины памяти видеокарты;
- coolingType – тип системы охлаждения видеокарты;
- powerConnectors – количество разъёмов питания видеокарты;
- typePCIe – тип PCIe соединения видеокарты;
- videoOutput – типы видеовыходов, расположенных на задней панели видеокарты.
- Class RAM – оперативная память, специфические поля:
 - capacity – объём одного модуля;
 - frequency – частота оперативной памяти;
 - latency – тайминги (время доступа) оперативной памяти;
 - modulesCount – количество модулей в комплекте;
 - hasRGB – наличие или отсутствие RGB подсветки;
 - hasHeatSpreader – наличие или отсутствие у модуля радиатора охлаждения;
 - memoryType – тип оперативной памяти.
- Class Storage – постоянное запоминающее устройство (накопитель), специфические поля:
 - capacity – ёмкость накопителя;
 - storageType – тип накопителя;
 - readSpeed – скорость чтения накопителя;
 - writeSpeed – скорость записи накопителя;
 - formFactor – форм-фактор накопителя;
 - tbw – ресурс записи накопителя;
 - cacheSize – размер кэша накопителя;
 - nandType – тип памяти накопителя.
- Class PowerSupply – блок питания компьютера, специфические поля:
 - efficiency – сертификат энергоэффективности блока питания;
 - modularity – категория модульности блока питания;

- sataConnectors – количество SATA разъёмов блока питания;
 - pcieConnectors – количество PCIe разъёмов блока питания;
 - cpuConnectors – количество CPU разъёмов блока питания;
 - fanSize – размер кулера блока питания;
 - protections – поддерживаемые блоком питания категории защиты.
- Class Cooler – башенный кулер для охлаждения процессора или корпусный кулер компьютера, специфические поля:
- coolerType – тип кулера (башенный или корпусный);
 - socketCompatibility – поддерживаемые сокет (для башенного);
 - fanSize – размер вентилятора кулера;
 - fanSpeed – скорость вращения вентилятора кулера;
 - tdp – количество рассеиваемой кулером тепловой мощности;
 - heatPipes – количество тепловых трубок (для башенного);
 - hasRGB – отсутствие или наличие RGB подсветки;
 - hasPwm – наличие или отсутствие регулировки скорости вращения вентилятора кулера.
- Class Case – компьютерный корпус, специфические поля:
- caseType – тип корпуса;
 - motherboardFormFactor – доступные для размещения в корпусе размеры материнских плат;
 - driveBays35 – количество отсеков для размещения 3,5 см дисков;
 - driveBays25 – количество отсеков для размещения 2,5 см дисков;
 - frontPanel – разъёмы на передней панели компьютерного корпуса;
 - hasTemperedGlass – наличие или отсутствие закалённого стекла на боковой панели корпуса;
 - hasFanSupport – предусмотрены или нет места для установки кулеров в корпусе;
 - hasRGB – наличие или отсутствие RGB подсветки корпуса;
 - hasDustFilters – наличие или отсутствие пылевых фильтров в корпусе.

Каждый из перечисленных классов должен обладать набором методов для заполнения и передачи своих полей, то есть «сеттерами» и «геттерами», а также переопределённым методом `getFullInfoForClient`, для вывода полной информации о себе в консоль. Также, все эти классы агрегативно связаны с `ComputerShop`, ассоциативно с `FileManager` и `ProductManager`.

Класс `ProductManager` выполняет роль продуктового менеджера в компьютерном магазине. Не обладает какими-либо полями. Изначально весь его функционал должен был реализовываться в классе `Seller`, но для облегчения проектирования и разработки программы было принято решение - выделить отдельную сущность со следующими методами: `registrationCPU`, `printFullInformationCPU`, `rewriteCPU`, `printMiniInfo`, `searchProductForClient`, `searchProductForSeller`, `removeTovarAndProduct`, `buyProductTovar`; отвечающие соответственно за: регистрацию (добавление в каталог товаров) каждого из 12 типов выше описанных товаров (в примере для облегчения понимания UML-диаграммы указан только метод для процессоров, далее тоже самое и для остальных методов, чьё название оканчивается на `CPU`), загрузку из файла коллекции, содержащей набор товаров той или иной категории, с последующим выводом её содержимого в каталог товаров, изменение значений полей уже зарегистрированных товаров и их перезапись в файл, вывод минимального количества информации для той или иной категории товаров, размещаемой в интерфейсе программы, поиска товара по его ID для клиента и продавца, удаление из каталога товаров того или иного продукта, покупку с последующим списанием со склада и переносом в список совершённых покупок клиента того или иного товара. Класс `ProductManager` агрегатно связан с `ComputerShop` и ассоциативно со всеми классами товаров, а также с `FileManager`.

Класс `FileManager` представляет собой файловый менеджер, необходимый в проекте для работы с файлами, имеет следующие методы: `saveClient`, `loadClient`, `rewriteClient`; отвечающие соответственно за: запись

полученных коллекций с экземплярами тех или иных классов (в примере для облегчения понимания UML-диаграммы указан только метод для покупателей, далее тоже самое и для остальных методов), загрузку из соответствующего файла и последующую передачу коллекции, содержащей те или экземпляры вышеописанных классов), перезапись изменённых экземпляров классов. Для осуществления этих задач FileManager должен иметь в себе поля, содержащие пути до всех файлов с коллекциями объектов классов, в качестве примера на UML-диаграмме приведено поле CLIENTFILE. Данный класс агрегативно связан с ComputerShop и ассоциативно со всеми 12 классами вышеперечисленных товаров, а также с Seller и Client.

Разработанная диаграмма классов служит опорой для реализации программы. Основываясь на размещённых на ней классах, их методах, полях и взаимодействиях, будет построено конечное консольное приложение «Компьютерный магазин».

2.2 Реализация программы

Реализация программ обычно начинается с построения базовых классов. Разработка консольной программы «Компьютерный магазин» началась с создания класса User. На рисунке 2 приведён код класса User.

```
import java.io.Serializable;

public class User implements Serializable { 4 usages 2 inheritors 3 rtbn12 +1
    private static final long serialVersionUID = 2L; no usages

    private String name; 2 usages
    private String password; 2 usages
    private String login; 2 usages
    private int role; 2 usages

    public User(String name, String password, String login, int Role) { 2 usages 3 rtbn12
        this.name = name;
        this.password = password;
        this.login = login;
        this.role = Role; // 0 - Seller; 1 - Client
    }

    public String getName() { 2 usages 3 rtbn12
        return name;
    }

    public String getPassword() { 2 usages 3 rtbn12
        return password;
    }

    public String getLogin() { 1 usage 3 rtbn12
        return login;
    }

    public int getRole() { no usages 3 rtbn12
        return role;
    }
}
```

Рисунок 2 – Код класса User

Как и предполагалось в предыдущем пункте, данный класс реализует интерфейс Serializable для корректной работы процессов чтения и записи

экземпляров классов, наследующихся от него. Написаны простой конструктор, принимающий в качестве переменных значения, которые в нём же присваиваются полям класса, а также набор «геттеров», возвращающих их значение. Все поля имеют класс доступа `private`, а методы – `public`, что отражает один из принципов объектно-ориентированного программирования – инкапсуляцию.

Далее был разработан класс `Seller`. Код данного класса представлен на рисунке 3.

```
import java.io.Serializable;
import java.util.InputMismatchException;
import java.util.Scanner;

public class Seller extends User implements Serializable { 8 usages  rtbn12

    private static final long serialVersionUID = 2L; no usages
    private String sellerKey; 1 usage
    private static transient Scanner scanner2 = new Scanner(System.in); 5 usages

    public Seller(String name, String password, String login, int Role, String sellerKey) { 1 usage  rtbn12
        super(name, password, login, Role 0);

        this.sellerKey = sellerKey;
    }

    public void initScanner() 1 usage  rtbn12
    {
        if(scanner2 == null)
        {
            scanner2 = new Scanner(System.in);
        }
    }

    public void registrationProduct() 2 usages  rtbn12
    {...}

}
```

Рисунок 3 – Код класса `Seller`

Данный класс, являясь наследником `User`, реализует ещё один принцип объектно-ориентированного программирования – наследование. Помимо уже упомянутого ранее поля `sellerKey`, хранящего в себе значение специального ключа, необходимого для регистрации нового продавца, для работы метода `registrationProduct` было добавлено новое поле `scanner2` типа данных `Scanner`,

настроенного на стандартный поток ввода. Как оказалось в процессе дальнейшей разработки программы, объект с таким полем не может корректно пройти процесс сериализации и записаться в файл, в следствие чего к названию переменной `scanner2` добавилось дополнительное слово – `transient`. Если написать это слово при формировании поля, то его значение при процессе сериализации будет записываться как «`null`». Другими словами, при записи объекта класса `seller` в файл, значение поле `scanner2` автоматически примет нулевое значение. При загрузке такого объекта из файла значение поля не меняется, то есть `scanner2` продолжает иметь нулевое значение, из-за чего не будет работать метод `registrationProduct`. Для решения этой проблемы был разработан метод `initScanner`, который проверяет, равно ли значение поля `scanner2` `null`, в случае эквивалентности – переопределяет его на новый стандартный поток ввода. У класса `Seller` есть классический конструктор с `super`-конструктором внутри. Метод `registrationProduct` предоставляет продавцу возможность добавить в каталог товаров продукт любой категории, в зависимости от введённого им числа, благодаря применению `try-catch` блока становится возможным отлавливать и обрабатывать ошибки, связанные с некорректным вводом варианта.

Следующим разработанным наследником класса `User` является `Client`. Частичный код данного класса представлен на рисунке 4. Все запланированные на моменте проектирования поля и методы были успешно реализованы. В качестве коллекции для хранения списка покупок, совершённых клиентом компьютерного магазина, был выбран классический `ArrayList`. В методе для вывода содержимого списка покупок в консоль, то есть в `printShoppingList`, код которого представлен на рисунке 5, предусмотрена предварительная его сортировка по значению поля `payDate` (дата покупки) класса `Product`. Для хранения товаров в корзине клиента была выбрана коллекция `LinkedList`, в которой автоматически запоми-

```

import java.util.*;

public class Client extends User{ 11 usages  Egor Solodov +1

    private double balance; 6 usages
    private List<Product> shoppingList; 7 usages
    private LinkedList<Product> shoppingCart; 24 usages

    public Client(String name, String password, String login, int Role) { 1 usage  rtbn12 +1
        super(name, password, login, Role);
        this.balance = 0;
        shoppingList = new ArrayList<>();
        shoppingCart = new LinkedList<>();
    }

    public double getBalance() { return this.balance; }

    public void setBalance(double balance) { this.balance = balance; }

    public void addBalance(double money) { this.balance += money; }

    public void deductBalance(double money) 1 usage  rtbn12
    {...}

    public void addShoppingList(Product product) { shoppingList.add(product); }

    public void printShoppingList() 1 usage  rtbn12 +1
    {...}

    public void addShoppingCart(Product product){...}

    public void printShoppingCart(){...}

    public void clearShoppingCart(){...}

    public List<Product> getShoppingList() { return shoppingList; }

    public void setShoppingList(List<Product> shoppingList) { this.shoppingList = shoppingList; }

    public LinkedList<Product> getShoppingCart() { return shoppingCart; }

    public void setShoppingCart(LinkedList<Product> shoppingCart) { this.shoppingCart = shoppingCart; }
}

```

Рисунок 4 – Код класса Client

```

public void printShoppingList() 1 usage  rtbn12 +1
{
    if(!shoppingList.isEmpty()){
        shoppingList.sort(Comparator.comparing(Product::getPayDate).reversed());

        for(Product product : shoppingList){

            product.getMiniInfo();
            product.getFullInformationForShoppingList();
            System.out.println("\n=====");
        }
    }
    else {
        System.out.println("\nВаш список покупок пока что пуст!");
    }
}

```

Рисунок 5 – Код метода printShoppingList

-нается порядок добавляемых в него продуктов. Интересным с точки зрения реализации является метод reFresh, код которого представлен на рисунке 6.

```
public void reFresh() { 3 usages Egor Solodov
    if (shoppingCart.isEmpty()) return;

    Map<String, Product> products = FileManager.loadProduct0();
    LinkedList<Product> newCart = new LinkedList<>();

    for (Product product : shoppingCart) {
        Product freshProduct = products.get(product.getId());

        if (freshProduct != null && freshProduct.getQuantityProduct() > 0) {
            newCart.add(freshProduct);
        } else {
            System.out.println("\nТовар \"" + product.getBrand() + " " +
                product.getModel() + "\" " +
                (freshProduct == null ? "удален из каталога" : "закончился") +
                " и удален из корзины!");
        }
    }

    shoppingCart.clear();
    shoppingCart.addAll(newCart);

    System.out.println("\nКорзина обновлена!");
}
```

Рисунок 6 – Код метода reFresh

Далее будет показано, что при входе покупателя в раздел со своей корзиной или при покупке товаров из неё этот метод автоматически запускается. В случае, если корзина не пуста происходит загрузка словаря (коллекции Map) со всеми продуктами, содержащимися в каталоге товаров магазина, и создаётся новая корзина, далее в цикле «for - each» идёт проверка на существование в данный момент в каталоге товаров каждого продукта в корзине клиента, а так же их количество на складе, и в случае, если товар в корзине существует в каталоге товаров и есть на складе, то он добавляется в

новую корзину, при несоблюдении хоть одного из двух условий – нет! После старого содержимое корзины клиента очищается и заполняется продуктами, прошедшими проверку. Этот метод важен, так как не позволяет купить из корзины товар, который перестал существовать в каталоге или отсутствует на складе, синхронизируя информацию с актуальным перечнем продуктов, что позволяет так же всегда обновлять состояние товаров в корзине покупателя. Например, при изменении цены на продукт продавцом, его цена изменится и в корзинах всех клиентов, которые авторизовались и зашли в соответствующий отдел.

Следующий разработанный класс – AccountManager, имеет, как и планировалось, 4 метода: для регистрации продавцов, покупателей, для авторизации продавцов, покупателей. Сокращённый код данного класса представлен на рисунке 7.

```
import java.util.Map;
import java.util.Scanner;

public class AccountManager { 5 usages  & rtbn12

    private static final String sellerKey = "BSU"; 1 usage

    public static Client authorizationClient(){...}

    public static void registrationClient() {...}

    public static Seller authorizationSeller(){...}

    public static void registrationSeller(){...}
```

Рисунок 7 – Код класса AccountManager

Процесс авторизации клиента и продавца схожи, но отличаются появляющимся в консоли текстом, а также вызываемыми методами класса FileManager. На рисунке 8 представлен код метода authorizationSeller.

```

public static Seller authorizationSeller(){ 1 usage  rtbn12
    Scanner scanner = new Scanner(System.in);
    Map<String,Seller> sellers = FileManager.loadSeller();

    System.out.print("\nПриветствую и желаю хорошего рабочего дня, дорогой Сотрудник!\n" +
        "Сейчас вам предстоит ввести ваши ранее придуманные логин и пароль!\n" +
        "Введите ваш логин:");
    String login = scanner.nextLine();

    while (!sellers.containsKey(login))
    {
        System.out.print("\nСотрудник с таким логином пока не зарегистрирован!\n" +
            "Попробуйте ввести ваш логин снова!");
        login = scanner.nextLine();
    }

    Seller seller = sellers.get(login);

    System.out.print("\nВведите ваш пароль:");
    String password = scanner.nextLine();
    while (!password.equals(seller.getPassword()))
    {
        System.out.print("\nВы ввели неверный пароль!\n" +
            "Попробуйте снова!");
        password = scanner.nextLine();
    }

    System.out.print("\nАвторизация прошла успешно!\n" +
        "Ещё раз желаю вам, " + seller.getName() + ", хорошего рабочего дня!\n" +
        "Ваш '" + ComputerShop.getName() + "' !");

    return seller;
}

```

Рисунок 8 – Код метода authorizationSeller

В начале в консоли появляется приветственное сообщение с просьбой правильно ввести ранее придуманные логин и пароль, затем зарегистрированный продавец выполняет эти действия, пока не введёт корректные значения - процесс авторизации не будет закончен. Для проверки вводимых логина и пароля при помощи метода loadSeller класса FileManager загружается словарь (коллекция Map), где ключом выступает логин продавца, а значением — объект класса Seller с таким же логином. После ввода пользователем логина проходит проверка на наличие в коллекции такого ключа, в случае нахождения создаётся объект класса Seller, принимающий значение зарегистрированного продавца. Проверка пароля происходит

сравнением введенного пользователем значения и уже заложенной информации в поле password полученного объекта зарегистрированного продавца. После прохождения всех проверок authorizationSeller возвращает объект зарегистрированного продавца в метод main класса ComputerShop, после чего процесс авторизации считается оконченным.

Процесс регистрации для покупателей и продавцов отличается одной дополнительной проверкой для последних. Код метода registrationSeller представлен на рисунке 9.

```
public static void registrationSeller(){ 1 usage 8 rtbn12
    Scanner scanner = new Scanner(System.in);
    Map<String, Seller> sellers = FileManager.loadSeller();

    System.out.print("\nПриветствуем вас в компании '" + ComputerShop.getName() + "'!\n" +
        "Мы ценим каждого сотрудника и очень рады от того, что вы решили присоединиться к нашей дружной семье!\n" +
        "Сейчас вам предстоит Ввести ваше НАСТОЯЩЕЕ имя, а также придумать и ввести логин и пароль, \n" +
        "а также специальное кодовое слово (вам его сообщил действующий сотрудник)!\n" +
        "Для успешного прохождения регистрации значение пароля при первичном и вторичном вводе должны быть идентичны!\n" +
        "Введите ваше ИМЯ:");
    String name = scanner.nextLine();

    System.out.print("\nПридумайте и введите ваш ЛОГИН:");
    String login = scanner.nextLine();
    while (sellers.containsKey(login))
    {
        System.out.print("\nТакой логин уже занят!\n" +
            "Пожалуйста, придумайте и введите другой:");
        login = scanner.nextLine();
    }

    System.out.print("\nПридумайте и введите ваш ПАРОЛЬ:");
    String password = scanner.nextLine();

    System.out.print("\nВведите пароль ещё раз:");
    String password2 = scanner.nextLine();

    while(!password2.equals(password))
    {
        System.out.print("\nПервичный пароль не соответствует вторичному!\n" +
            "Попробуйте снова:");
        password2 = scanner.nextLine();
    }

    System.out.print("\nПароли совпали!");

    System.out.print("\nВведите кодовое слово:");
    String sellerKey0 = scanner.nextLine();
    while (!sellerKey0.equals(sellerKey))
    {
        System.out.print("\nВы ввели неверное кодовое слово!\n" +
            "Попробуйте ещё раз!");
        sellerKey0 = scanner.nextLine();
    }

    System.out.print("\nРегистрация прошла успешно!\n" +
        "Спасибо за то, что выбрали работать в нашей компании!");

    sellers.put(login, new Seller(name,password,login, Role: 0, sellerKey0));
    FileManager.saveSeller(sellers);
}
```

Рисунок 9 – Код метода registrationSeller

В начале происходит процесс получения коллекции со всеми зарегистрированными продавцами, идентичный тому, что был описан в методе `authorizationSeller`. После чего в консоль выводится приветственное сообщение о том, что администрация компьютерного магазина ценит всех своих сотрудников, для продолжения регистрации следует ввести логин и пароль. Для того что бы продавец не регистрировался под уже занятым логином, происходит проверка на наличие введённого им значения среди ключей в коллекции со всеми зарегистрированными продавцами. Процесс регистрации не продолжится, пока новый продавец не придумает и введёт уникальный новый логин. Пароль же, как это принято, вводится новым продавцом дважды, и пока второе введённое значения пароля не совпадёт с первым – процесс регистрации не продолжится. Вся описанная последовательность действий схожим образом реализована и для покупателей в методе `registrationClient`. Но в методе для продавцов есть дополнительное условия для успешного прохождения регистрации – ввод кодового слова. Значение этого слова может поменяться в любой момент времени, достаточно изменить значение поля `sellerKey` описываемого класса. На данный момент, значение кодового слова – «BSU», что расшифровывается как «Belgorod State University — Белгородский государственный университет». Пока новый продавец не введёт именно это значение, регистрация не будет закончена. Данный шаг был сделан с целью ограничения возможности зарегистрироваться как продавец всем желающим, так как `Seller` обладает полной властью над каталогом товаров компьютерного магазина, недоброжелатель может, например, поставить околонулевую цену всем продуктам или просто удалить весь каталог, другими словами, ввод кодового слова в процессе регистрации – простейшая форма защиты магазина. После выполнения всех условий в коллекцию со всеми продавцами добавляется новый, а сама коллекция при помощи метода `saveSeller` класса `FileManager` загружается для долговременного хранения в соответствующий файл. Процесс регистрации на данном моменте окончен.

Все методы и поле AccountManager статические, то есть принадлежат самому классу, а не его объектам. Сделано это потому, что функционал этого класса понадобятся позже в main, ComputerShop, и чтобы там не создавать каждый раз экземпляры, при такой реализации достаточно просто прописывать, например «AccountManager.AutorizationClient» и автоматически запустится необходимая функция. Схожее решение будет неоднократно применено в программе.

После был написан класс Product, все запланированные методы и поля были успешно реализованы. Сокращённый код данного класса приведён на рисунке 10.

```
import java.io.Serializable;
import java.util.Date;
import java.util.Scanner;

public class Product implements Serializable { 81 usages 14 inheritors  rtbn12 +1

    private static final long serialVersionUID = 1L; no usages

    private final String id; 3 usages
    private String brand; 4 usages
    private String model; 4 usages
    private String countryProduction; 3 usages
    private int productionDate; 3 usages
    private int price; 3 usages
    private String category; 3 usages
    private String description; 3 usages
    private Date addedDate; 4 usages
    private double rating; 3 usages
    private int warrantyMoths; 3 usages
    private int quantityProduct; 4 usages
    private Date payDate; 4 usages

    public Product(String id, String category, Scanner scanner) {...}
    public Product(String id, String brand, String model, no usages  rtbn12
        String countryProduction, int productionDate,
        int price, String category, String description,
        Date addedDate, double rating, int warrantyMoths,
        int quantityProduct, Date payDate) {...}

    public String getId() { return id; }
    public String getModel() { return model; }
    public String getBrand() { return brand; }
    public String getCountryProduction() { return countryProduction; }
```

Рисунок 10 – Код класса Product

Как и было задумано на этапе проектирования, все поля класса имеют класс доступа «private», то есть получить к ним доступ без соответствующего

«геттера» вне самого класса невозможно, представляют собой общие характеристики для любого товара из 12 доступных категорий.

Product имеет 2 конструктора, код конструкторов представлен на рисунке 11.

```
public Product(String id, String category, Scanner scanner) { 3 usages 8 rtbn12
    this.id = id;
    this.addedDate = new Date();
    this.payDate = new Date();
    setBrand(scanner);
    setModel(scanner);
    setCountryProduction(scanner);
    setProductionDate(scanner);
    setPrice(scanner);
    setCategory(category);
    setDescription(scanner);
    setWarrantyMoths(scanner);
    setQuantityProduct(scanner);
    setRating(scanner);
}
public Product(String id, String brand, String model, no usages 8 rtbn12
    String countryProduction, int productionDate,
    int price, String category, String description,
    Date addedDate, double rating, int warrantyMoths,
    int quantityProduct, Date payDate) {
    this.id = id;
    this.brand = brand;
    this.model = model;
    this.countryProduction = countryProduction;
    this.productionDate = productionDate;
    this.price = price;
    this.category = category;
    this.description = description;
    this.addedDate = addedDate;
    this.rating = rating;
    this.warrantyMoths = warrantyMoths;
    this.quantityProduct = quantityProduct;
    this.payDate = payDate;
}
```

Рисунок 11 – Код конструкторов класса Product

Нижний – классический вариант, то есть в качестве параметров передаются все данные, необходимые для заполнения всех полей. Он использовался для осуществления процесса регистрации нового продукта в

первых вариантах реализации ответственных за это действие методах класса ProductManager. Идея заключалась в том, чтобы осуществлять проверку корректности вводимых значений для полей класса Product и его наследников (далее этот процесс может называться «валидацией») в отдельных методах для каждого класса. Такой подход казался весьма удобным, так как можно было менять условия для проверки и не бояться, что придётся заново переписывать все уже созданные экземпляры классов. Но у такого подхода был один существенный недостаток. Как оказалось, такие методы получались весьма объёмными и весь код класса ProductManager (ответственного за добавления новых товаров в каталог, их изменения и удаление) составлял более 5000 строк, только с реализованной регистрацией, но, когда настал момент реализовывать методы для изменения полей для каждой категории товаров, стало ясно, все эти 5000 строк будут в общем итоге просто продублированы, что является неудобной и непрофессиональной практикой, вследствие чего было принято решение переработать логику заполнения полей как для класса Product, так и для его наследников (Изменения, внесённые в методы класса ProductManager, будут перечислены при непосредственном описании реализации класса).

Из-за чего и был разработан новый конструктор (на рисунке 11 он изображён сверху). Он принимает только три переменных: ID регистрируемого товара, категорию будущего продукта (при добавлении товара в каталог магазина создаются как экземпляр конкретного класса, например CPU, так и класса Product, обоснование этого решения уже было приведено ранее) и объект класса Scanner. Для поля id присваивается переданное в качестве параметра значение, для поля addedDate, несущего в себе дату добавления товара в каталог, логично автоматически присваивается значение текущего реального времени, для поля payDate (время покупки) временно присваивается тоже значение. Далее происходит автоматический вызов «сеттеров» для заполнения всех оставшихся полей, именно в них была перенесена логика валидации. Не сложно догадаться, что для всех полей была

разработана своя пара «getter-setter». На рисунке 12 приведён пример кода разработанной пары для поля quantityProduct (количество штук товара, поступивших или уже находящихся на складе).

```
public int getQuantityProduct() {  rtbn12
    return quantityProduct;
}
public void setQuantityProduct(Scanner scanner) {  13 usages  rtbn12
    boolean valid = false;
    while (!valid) {
        try {
            System.out.print("\nВведите количество единиц товара, поступающего на склад: ");
            int input = scanner.nextInt();
            scanner.nextLine(); // очистка буфера

            if (input < 0 || input > 10) {
                System.out.println("Некорректное количество товара! (0-10)");
                continue;
            }
            this.quantityProduct = input;
            valid = true;
        } catch (java.util.InputMismatchException e) {
            System.out.println("Произошла ошибка!\n" +
                "Пожалуйста, введите корректное целочисленное значение!\n" +
                "В прошлый раз вы ввели букву вместо числа!");
            scanner.nextLine();
        } catch (Exception e) {
            System.out.println("Произошла неизвестная ошибка!");
            scanner.nextLine();
        }
    }
}
```

Рисунок 12 – Код методов getQuantityProduct и setQuantityProduct

getQuantityProduct – классический «getter», как и все остальные у всех классов наследников Product. setQuantityProduct – «setter» со встроенной валидацией заполняемого поля, которая заключается в использовании цикла «while», блока для отслеживания и отработки ошибок ввода, он же «try-catch», выводе пояснительных предложений для любых возникающих штатных и нештатных ситуаций, а также условного блока if, в котором и проводится проверка на соответствие определённым параметрам для вводимого значения поля, в данном случае количество товаров, должно быть целым числом, большим или равным нулю и не превышающим 10 единиц. Такой подход практически полностью исключает возможность присвоения полю некорректного значения. Для работы этих методов необходимо в качестве

параметра передавать так называемые объекты «сканнеры» класса Scanner. Чтобы избежать всех тех трудностей, связанных со «сканнером», уже описанных в реализации класса Seller, и было принято решение просто передавать их в качестве параметра в конструктор. Аналогичные «сеттеры» были разработаны абсолютно для всех полей всех классов, представляющих собой продукты в каталоге товаров компьютерного магазина.

Кроме многочисленных пар методов «getter-сеттер» в классе Product были реализованы методы для вывода информации о значении полей в консоль для всех возможных для этого мест в программе. Код данных методов представлен на рисунке 13.

```
public void getMiniInfo() 10 usages  ↗ rtbn12
{
    System.out.println("\nId: " + getId());
    System.out.println("Категория: " + getCategory());
    System.out.println("Бренд: " + getBrand());
    System.out.println("Модель: " + getModel());
    System.out.println("Цена: " + getPrice());
    System.out.println("Рейтинг: " + getRating());
    System.out.println("Количество на складе: " + getQuantityProduct() + " шт.");
}
public void getFullInfoForClient() 12 overrides  ↗ rtbn12
{
}
public void getFullInfoForSeller() 24 usages  ↗ rtbn12 +1
{
    System.out.println("Дата добавления товара: " + getAddedDate());
}
public void getFullInformationForShoppingList() { System.out.println("Дата покупки товара: " + getPayDate()); }
```

Рисунок 13 – Код методов для вывода информации о классе Product

Как и было задумано на этапе планирования, в каталоге товаров компьютерного магазина изначально будет выводиться лишь минимальная необходимая информация о том или ином продукте. Для обеспечения этой задачи был разработан метод getMiniInfo, передающий в консоль информацию о ID товара, его категории, бренде, модели, цене, рейтинге и количестве на складе. Метод getFullInfoForClient в классе Product пуст, ведь он призван выводить полную информацию о товаре, как следует из названия – для клиента, но данный класс физически не имеет всех тех специфических полей,

присущих для остальных конкретных товаров, как и подразумевает принцип наследование, другими словами, данный метод разработан строго для наследников и будет в их коде «переопределяться». Метод `getFullInfoForSeller` дополнительно выводит в консоль информацию о дате поступления товара на склад, о ней покупателю знать вовсе не обязательно. Метод `getFullInformationForShoppingList` вызывается только в списке купленных товаров конкретного клиента и выводит в консоль информацию о дате покупки товара.

Иначе говоря, правильно написанный класс `Product`, является одним из самых значимых во всей программе, так как его логике подчиняются все его классы наследники.

Далее был разработан класс `Software`, объекты которого представляют собой конкретные единицы программного обеспечения, которые содержатся в каталоге товаров компьютерного магазина. Сокращенный код данного класса представлен на рисунке 14.

```
import java.util.List;
import java.util.Scanner;

public class Software extends Product { 19 usages  ⚡ rtbn12+1

    private String softwareType;        // Тип ПО (ОС, антивирус, офис, игра) 8 usages
    private int licenseDuration;        // Срок лицензии (месяцев, 0 = бессрочная) 2 usages
    private String supportedOS;         // Поддерживаемые ОС (Windows, macOS, Linux) 2 usages
    private String systemRequirements; // Системные требования 2 usages
    private boolean isDigital;          // Цифровая версия (true) или коробочная (false) 3 usages
    private String languages;           // Поддерживаемые языки 2 usages

    public Software(String id, String category, Scanner scanner) { 1 usage  ⚡ rtbn12
        super(id, category, scanner);
        setSoftwareType(scanner);
        setLicenseDuration(scanner);
        setSupportedOS(scanner);
        setSystemRequirements(scanner);
        setIsDigital(scanner);
        setLanguages(scanner);
    }

    public String getSoftwareType() { return softwareType; }
    public void setSoftwareType(Scanner scanner) {...}
    public int getLicenseDuration() { return licenseDuration; }
    public void setLicenseDuration(Scanner scanner) {...}
    public String getSupportedOS() { return supportedOS; }
    public void setSupportedOS(Scanner scanner) {...}
    public String getSystemRequirements() { return systemRequirements; }
    public void setSystemRequirements(Scanner scanner) {...}
    public boolean isDigital() { return isDigital; }
    public void setIsDigital(Scanner scanner) {...}
    public String getLanguages() { return languages; }
    public void setLanguages(Scanner scanner) {...}
    @Override ⚡ Egor Solodov +1
    public void getFullInfoForClient() {...}
}
```

Рисунок 14 – Код класса `Software`

Как и Product, данный класс имеет конструктор, принимающий только три уже описанных переменных, а также автоматически вызывающий при создании нового объекта набор «сеттеров», отвечающих за заполнение всех его полей. Все «сеттеры» Software не нарушают валидацию и не дают возможности присвоить полю класса не корректное значение. На рисунке 15 в качестве примера приведён код пары «getter-сеттер» для поля `licenseDuration`, хранящего в себе срок лицензии для конкретной единицы программного обеспечения.

```
public int getLicenseDuration() { 1 usage  Egor Solodov
    return licenseDuration;
}
public void setLicenseDuration(Scanner scanner) { 2 usages  rtbn12 +1
    boolean cycleLicenseDuration = true;
    while (cycleLicenseDuration) {
        System.out.print("\nВведите срок лицензии в месяцах (0 = бессрочная, 1-120 месяцев): ");

        try {
            int licenseDuration = scanner.nextInt();
            scanner.nextLine();

            while (licenseDuration < 0 || licenseDuration > 120) {
                System.out.print("\nНекорректный срок лицензии!\n" +
                    "Введите значение ещё раз (0-120 месяцев): ");
                licenseDuration = scanner.nextInt();
                scanner.nextLine();
            }

            this.licenseDuration = licenseDuration;
            cycleLicenseDuration = false;

        } catch (java.util.InputMismatchException p) {
            System.out.println("Произошла ошибка!\n" +
                "Пожалуйста, введите корректное значение!\n" +
                "В прошлый раз вы ввели букву вместо числа!");
            scanner.nextLine();
        } catch (Exception p) {
            System.out.println("Произошла неизвестная ошибка!");
            scanner.nextLine();
        }
    }
}
```

Рисунок 15 – Код методов `getLicenseDuration` и `setLicenseDuration`

В данном случае корректным будет считаться срок лицензии от нуля, до 10 лет, где вариант ноль обозначает бессрочную лицензию.

В данном классе, как и во всех классах, обозначающих конкретный продукт каталога товаров компьютерного магазина, переопределён метод `getFullInfoForClient`. Код метода представлен на рисунке 16.

```
@Override  Egor Solodov +1
public void getFullInfoForClient() {
    System.out.println("\nId: " + getId());
    System.out.println("Категория: " + getCategory());
    System.out.println("Бренд: " + getBrand());
    System.out.println("Модель: " + getModel());
    System.out.println("Тип программного обеспечения: " + getSoftwareType());
    System.out.println("Срок лицензирования: " + getLicenseDuration() + " месяцев");
    System.out.println("Поддерживаемые операционные системы: " + getSupportedOS());
    System.out.println("Системные требования: " + getSystemRequirements());
    System.out.println("Цифровая версия: " + (isDigital()? "Да" : "Нет"));
    System.out.println("Поддерживаемые языки: " + getLanguages());
    System.out.println("Цена: " + getPrice() + " рублей");
    System.out.println("Страна-производитель: " + getCountryProduction());
    System.out.println("Год релиза: " + getProductionDate());
    System.out.println("Гарантийный срок: " + getWarrantyMonths() + " месяцев");
    System.out.println("Рейтинг: " + getRating() + " из 5!");
    System.out.println("Количество на складе: " + getQuantityProduct() + " шт.");
    System.out.println("Описание: " + getDescription());
}
```

Рисунок 16 – Код метода `getFullInfoForClient`

Как и задумывалось на этапе планирования, данный метод при его вызове выводит полную информацию о программном обеспечении в консоль.

Следующим бал разработан класс `ComputerComponent`, олицетворяющий собой абстрактную компьютерную деталь, данный класс служит посредником между `Product` и конкретным компьютерным компонентом, например процессором. В следствии чего, объекты такого класса не будут создаваться, а сам он несёт только те поля, какие могут быть у любого наследника. По описанным причинам не переопределяет метод `getFullInfoForClient`, содержит только пары «геттер-сеттер» для работы со своими полями. Сокращённый код `ComputerComponent` представлен на рисунке 17.


```

import java.util.Scanner;

public class ComputerComponent extends Product { 8 usages 8 inheritors 8 rtbn12

    private String interfaceType; 13 usages
    private int powerConsumption; 2 usages
    private double voltage; 2 usages

    public ComputerComponent(String id, String category, Scanner scanner) { 8 usages 8 rtbn12
        super(id, category, scanner);
        setInterfaceType(scanner, category);
        setPowerConsumption(scanner);
        setVoltage(scanner);
    }

    public String getInterfaceType() { return interfaceType; }
    public int getPowerConsumption() { return powerConsumption; }
    public double getVoltage() { return voltage; }
    public void setInterfaceType(Scanner scanner, String category) {...}
    public void setPowerConsumption(Scanner scanner) {...}
    public void setVoltage(Scanner scanner) {...}
}

```

Рисунок 17 – Код класса ComputerComponent

Далее был разработан схожий по своей логике с ComputerComponent класс Peripheral, представляющей собой абстрактное периферийное устройство. Данный класс, как и ComputerComponent не переопределяет getFullInfoForClient, содержит только пары «геттер-сеттер» для работы со своими полями. Сокращённый код Peripheral представлен на рисунке 18.

```

import java.util.Scanner;

public class Peripheral extends Product { 3 usages 3 inheritors 8 rtbn12 +1

    private String connectionType; // Тип подключения (USB, Bluetooth, Wireless) 8 usages
    private double cableLength; // Длина кабеля (м), 0 для беспроводных 3 usages
    private boolean isWireless; // Беспроводное устройство 4 usages
    private boolean hasRGB; // RGB подсветка 3 usages
    private String color; // цвет устройства 8 usages

    public Peripheral(String id, String category, Scanner scanner) { 3 usages 8 rtbn12
        super(id, category, scanner);
        setConnectionType(scanner);
        setIsWireless(scanner);
        setCableLength(scanner);
        setHasRGB(scanner);
        setColor(scanner);
    }

    public String getConnectionType() { return connectionType; }
    public void setConnectionType(Scanner scanner) {...}
    public double getCableLength() { return cableLength; }
    public void setCableLength(Scanner scanner) {...}
    public boolean isWireless() { return isWireless; }
    public void setIsWireless(Scanner scanner) {...}
    public boolean isHasRGB() { return hasRGB; }
    public void setHasRGB(Scanner scanner) {...}
    public String getColor() { return color; }
    public void setColor(Scanner scanner) {...}
}

```

Рисунок 18 – Код класса Peripheral

Следующими были разработаны 8 классов-наследников ComputerComponent. Первый из них – CPU, олицетворяющий собой центральный процессор компьютера. Сокращённый код данного класса представлен на рисунке 19.

```
import java.util.Scanner;

public class CPU extends ComputerComponent { 19 usages  ⚡ rtbn12
    private String socket;                // AM4, LGA1700, AM5 2 usages
    private int coreCount;                // Количество ядер (6, 8, 12) 2 usages
    private int threadCount;              // Количество потоков (12, 16, 24) 2 usages
    private double baseFrequency;         // Базовая частота (ГГц) 3 usages
    private double maxFrequency;          // Максимальная частота (ГГц) 2 usages
    private int cacheSize;                // Размер кэша (МБ) 2 usages
    private String memoryType;            // Поддерживаемая память (DDR4, DDR5) 2 usages
    private boolean hasIntegratedGraphics; // Встроенная графика 4 usages

    public CPU(String id, String category, Scanner scanner) { 1 usage  ⚡ rtbn12
        super(id, category, scanner);
        setSocket(scanner);
        setCoreCount(scanner);
        setThreadCount(scanner);
        setBaseFrequency(scanner);
        setMaxFrequency(scanner);
        setCacheSize(scanner);
        setMemoryType(scanner);
        setHasIntegratedGraphics(scanner);
    }

    public String getSocket() { return socket; }
    public void setSocket(Scanner scanner) {...}
    public int getCoreCount() { return coreCount; }
    public void setCoreCount(Scanner scanner) {...}
    public int getThreadCount() { return threadCount; }
    public void setThreadCount(Scanner scanner) {...}
    public double getBaseFrequency() { return baseFrequency; }
    public void setBaseFrequency(Scanner scanner) {...}
    public double getMaxFrequency() { return maxFrequency; }
    public void setMaxFrequency(Scanner scanner) {...}
    public int getCacheSize() { return cacheSize; }
    public void setCacheSize(Scanner scanner) {...}
    public String getMemoryType() { return memoryType; }
    public void setMemoryType(Scanner scanner) {...}
    public boolean isHasIntegratedGraphics() { return hasIntegratedGraphics; }
    public void setHasIntegratedGraphics(Scanner scanner) {...}
    @Override  ⚡ rtbn12
    public void getFullInfoForClient() {...}
}
```

Рисунок 19 – Код класса CPU

Объекты класса CPU, как и Software, представляют собой конкретные единицы товаров, в следствие чего будут добавляться в каталог компьютерного магазина, в данном классе переопределён метод getFullInfoForClient, код представлен на рисунке 20.

```

@Override
public void getFullInfoForClient() {
    System.out.println("\nId: " + getId());
    System.out.println("Категория: " + getCategory());
    System.out.println("Бренд: " + getBrand());
    System.out.println("Модель: " + getModel());
    System.out.println("Цена: " + getPrice() + " рублей");
    System.out.println("Интерфейс подключения: " + getInterfaceType());
    System.out.println("Сокет: " + getSocket());
    System.out.println("Количество ядер: " + getCoreCount());
    System.out.println("Количество потоков: " + getThreadCount());
    System.out.println("Базовая частота: " + getBaseFrequency() + " ГГц");
    System.out.println("Максимальная частота: " + getMaxFrequency() + " ГГц");
    System.out.println("Размер кэша: " + getCacheSize() + " МБ");
    System.out.println("Поддерживаемые типы памяти: " + getMemoryType());
    System.out.println("Наличие встроенного графического ядра: " + (hasIntegratedGraphics?" Да":" Нет"));
    System.out.println("Потребляемая мощность: " + getPowerConsumption() + " Вт");
    System.out.println("Рабочее напряжение: " + getVoltage() + " В");
    System.out.println("Страна-производитель: " + getCountryProduction());
    System.out.println("Год релиза: " + getProductionDate());
    System.out.println("Гарантийный срок: " + getWarrantyMonths() + " месяцев");
    System.out.println("Рейтинг: " + getRating() + " из 5!");
    System.out.println("Количество на складе: " + getQuantityProduct() + " шт.");
    System.out.println("Описание: " + getDescription());
}

```

Рисунок 20 – Код метода getFullInfoForClient

По схожей логике реализованы остальные 7 классов-наследников ComputerComponent и 3 класса-наследника Peripheral: GPU, MotherBoard, RAM, Storage, PowerSupply, Cooler, Case, Monitor, Keyboard, Mouse.

Очень важным шагом является разработка класса ProductManager, методы которого осуществляют непосредственное добавление, удаление, изменение, поиск продуктов из каталога товаров компьютерного магазина, их покупку и вывод информации о них. Сокращенный код данного класса представлен на рисунке 21.

Проанализировав рисунок, можно заметить, что методы класса отделены пробелами друг от друга, образуя 6 разных групп, в каждой из которой находятся функции, выполняющие схожие задачи. В качестве примера, будет проведено описание только одного метода из каждой группы.

```

import java.util.*;

public class ProductManager { 37 usages Egor Solodov +1

    public static void registrationCPU(String choiceCategory) {...}
    public static void registrationMotherBoard(String choiceCategory) {...}
    public static void registrationGPU(String choiceCategory){...}
    public static void registrationRAM(String choiceCategory) {...}
    public static void registrationStorage(String choiceCategory) {...}
    public static void registrationPowerSupply(String choiceCategory){...}
    public static void registrationCase(String choiceCategory){...}
    public static void registrationCooler(String choiceCategory){...}
    public static void registrationSoftware(String choiceCategory){...}
    public static void registrationMonitor(String choiceCategory){...}
    public static void registrationMouse(String choiceCategory){...}
    public static void registrationKeyBoard(String choiceCategory){...}

    public static Map<String,Product> printMiniInfo(String category){...}
    public static Product searchProductForClient(){...}
    public static Product searchProductForSeller(){...}

    public static void printFullInformationCPU(){...}
    public static void printFullInformationRAM() {...}
    public static void printFullInformationMotherboard() {...}
    public static void printFullInformationGPU() {...}
    public static void printFullInformationStorage() {...}
    public static void printFullInformationPowerSupply() {...}
    public static void printFullInformationCase() {...}
    public static void printFullInformationCooler() {...}
    public static void printFullInformationSoftware() {...}
    public static void printFullInformationMonitor() {...}
    public static void printFullInformationMouse() {...}
    public static void printFullInformationKeyboard() {...}

    public static void removeTovarAndProduct(Scanner scanner){...}

    public static void rewriteCPU(String id, Scanner scanner){...}
    public static void rewriteMotherBoard(String id, Scanner scanner) {...}
    public static void rewriteGPU(String id, Scanner scanner) {...}
    public static void rewriteRAM(String id, Scanner scanner) {...}
    public static void rewriteStorage(String id, Scanner scanner) {...}
    public static void rewritePowerSupply(String id, Scanner scanner) {...}
    public static void rewriteCase(String id, Scanner scanner) {...}
    public static void rewriteCooler(String id, Scanner scanner) {...}
    public static void rewriteSoftware(String id, Scanner scanner) {...}
    public static void rewriteMonitor(String id, Scanner scanner) {...}
    public static void rewriteMouse(String id, Scanner scanner) {...}
    public static void rewriteKeyboard(String id, Scanner scanner) {...}
    public static void reWriteTovarAndProduct(Scanner scanner){...}

    public static boolean buyProductTovar(Product product,Client client, Scanner scanner){...}

}

```

Рисунок 21 – Код класса ProductManger

Первая группа – методы для регистрации (добавления в каталог) новых товаров, именно они используются в методе registrationProduct класса Seller.

Код метода registrationCPU представлен на рисунке 22.

```
public static void registrationCPU(String choiceCategory) {
    Scanner scanner = new Scanner(System.in);
    Map<String, Product> products = FileManager.loadProduct0();

    try {
        System.out.println("\n=== Регистрация процессора ===");

        System.out.print("\nВведите ID продукта: ");
        String id = scanner.nextLine().toUpperCase().trim();

        while (products.containsKey(id)) {
            System.out.print("\nТакой ID уже занят!\nПридумайте другой: ");
            id = scanner.nextLine().toUpperCase().trim();
        }

        Map<String, CPU> cpus = FileManager.loadCPU();

        CPU cpu = new CPU(id, choiceCategory, scanner);

        products.put(id, cpu);
        FileManager.saveProduct0(products);

        cpus.put(id, cpu);
        FileManager.saveCPU(cpus);

        System.out.print("\nРегистрация товара завершена успешно!");
    } catch (Exception e) {
        System.out.println("\nОшибка при регистрации процессора: " + e.getMessage());
    }
}
```

Рисунок 22 – Код метода registrationCPU

Изначально при помощи статического метода loadProduct0 класса FileManager происходит загрузка коллекции products, в которой содержатся все зарегистрированные на данный момент экземпляры класса Product. Далее, происходит ввод ID нового товара и проверка на то, чтобы вводимый продавцом ID не принадлежал другим товарам. Проверка осуществляется путём поиска в загруженной коллекции ID - ключа, идентичному тому, что ввёл продавец, пока последний не введёт новый и уникальный, процесс регистрации не продолжится. После, при помощи статического метода loadCPU класса FileManager происходит загрузка коллекции cpus, в которой записаны все зарегистрированные процессоры. Далее, создаётся новый процессор, в конструктор которого заносятся введённое ID, категория товара,

которая передаётся в метод в качестве параметра, и объект класса Scanner. После чего происходит последовательные добавление нового процессора в коллекции products и cpus и сохранение этих коллекций при помощи статических методов класса FileManager saveProduct() и saveCPU() соответственно. На этом процесс регистрации нового процессора окончен.

На рисунке 23 представлен код метода printMiniInfo.

```
public static Map<String,Product> printMiniInfo(String category){ 2 usages  ⚙ rtbn12

    Map<String,Product> products = FileManager.loadProduct0();
    System.out.println("===== Базовая информация =====");
    for (Product product : products.values())
    {
        if (product.getCategory().equals(category)) {
            product.getMiniInfo();
            System.out.println("++++++");
        }
    }

    return products;
}
```

Рисунок 23 – Код метода printMiniInfo

Данный метод сначала загружает коллекцию со всеми продуктами из каталога товаров, а потом в зависимости от категории товара, передаваемой в качестве параметра метода, выводит в консоль базовую информацию о каждом из них, после чего возвращает эту коллекцию.

На рисунке 24 частично представлен код метода searchProductForClient. После загрузки коллекции со всеми продуктами, происходит ввод id товара, далее следует проверка, определяющая, имеется ли товар с таким id в коллекции или нет, если имеется, то у товара с таким id берётся категории, далее в зависимости от категории будут запускаться соответствующие методы для загрузки коллекций товаров этой категории, получение уже конкретного товара по тому же id и вывод полной информации о нём в консоль, в конце своей работы метод возвращает найденный товар.

```

public static Product searchProductForClient() { 3 usages  rtbn12 +1

    Scanner scanner = new Scanner(System.in);
    Map<String, Product> products = FileManager.loadProduct0();
    System.out.print("\nВведите id заинтересовавшего вас товара:");
    String id = scanner.nextLine();

    while(!products.containsKey(id)){
        System.out.print("\nТовара с таким id не найдено!\n" +
            "Пожалуйста, попробуйте ввести id товара ещё раз:");
        id = scanner.nextLine();
    }

    Product product = products.get(id);
    String category = product.getCategory();
    switch (category){

        case "Центральный процессор":
            Map<String,CPU> CPU = FileManager.loadCPU();
            CPU cpu = CPU.get(id);
            cpu.getFullInfoForClient();
            break;
        case "Материнская плата":
            Map<String,MotherBoard> MB = FileManager.loadMotherBoard();
            MotherBoard mb = MB.get(id);
            mb.getFullInfoForClient();
            break;
        case "Видеокарта":
            Map<String,GPU> GPU = FileManager.loadGPU();
            GPU gpu = GPU.get(id);
            gpu.getFullInfoForClient();
            break;
    }
}

```

Рисунок 24 – Код метода searchProductForClient

На рисунке 25 приведён код метода printFullInformationCPU.

```

public static void printFullInformationCPU() { 1 usage  rtbn12 +1

    Map<String, CPU> cpus = FileManager.loadCPU();

    System.out.println("\n+++++ Каталог процессоров +++++");

    if(!cpus.isEmpty()){
        for(CPU cpu : cpus.values()){
            cpu.getFullInfoForClient();
            cpu.getFullInfoForSeller();
            System.out.println("\n=====");
        }
    }
    else {
        System.out.println("\nТоваров данной категории пока что нет в магазине!");
    }
}

```

Рисунок 25 – Код метода printFullInformationCPU

Данный метод выполняет ту же последовательность действий, что и `printMiniInfo`, но выводит полную информацию о товаре, а также специальную информацию для продавца.

На рисунке 26 частично представлен код метода `removeTovarAndProduct`.

```
public static void removeTovarAndProduct(Scanner scanner){ 2 usages  ⚙ rtbn12 *
    int removeChoice;
    boolean removeCycle = true;
    while(removeCycle)
    {
        System.out.println("\n+++++ Удаления товара +++++");
        System.out.print("\nВыберите одно из доступных действий:\n" +
            "1 - Точно удалить товар\n" +
            "0 - Вернуться назад\n" +
            "Ваш выбор:");
        try {
            removeChoice = scanner.nextInt();
            scanner.nextLine();
            switch (removeChoice)
            {
                case 1:
                    int removeRemoveChoice;
                    boolean removeRemoveCycle = true;
                    while(removeRemoveCycle)
                    {
                        Map<String, Product> products = FileManager.loadProduct0();
                        System.out.print("\nВведите id товара, который будет удалён:");
                        String id = scanner.nextLine();

                        while(!products.containsKey(id)){
                            System.out.print("\nТовара с таким id не найдено!\n" +
                                "Пожалуйста, попробуйте ввести id товара ещё раз:");
                            id = scanner.nextLine();
                        }
                        boolean cycle = true;
                        while (cycle){
                            System.out.print("\nВы точно хотите продолжить удаление товара?\n" +
                                "ПОСЛЕДСТВИЯ БУДУТ НЕ ОБРАТИМЫ!\n" +
                                "Введите 1 - если хотите продолжить процесс удаления товара!\n" +
                                "Введите 0 - если хотите прервать процесс удаления!\n" +
                                "Ваш выбор:");
                            try {
                                removeRemoveChoice = scanner.nextInt();
                                scanner.nextLine();
                                switch (removeRemoveChoice)
                                {
                                    case 1:
                                        Product product = products.get(id);
                                        String category = product.getCategory();
                                        products.remove(id);
                                        FileManager.saveProduct0(products);
                                        switch (category){
                                            case "Центральный процессор":
                                                Map<String, CPU> CPUMap = FileManager.loadCPU();
                                                CPUMap.remove(id);
                                                FileManager.saveCPU(CPUMap);
                                                cycle = false;
                                                removeRemoveCycle = false;
                                                break;

```

Рисунок 26 – Код метода `removeTovarAndProduct`

В начале работы метода продавцу предстоит подтвердить своё намерение удалить товар, в зависимости от введенной цифры процесс удаления или продолжается, или прерывается. После продавцу следует ввести id товара, который он хочет удалить, после чего в консоль выводится полная информация о данном продукте. Далее продавцу предстоит сделать окончательный выбор о том, удалять продукт или нет, в случае, если тот полностью уверен в нужности сего действия, вне зависимости от категории товара, загружается коллекция со всеми продуктами, из неё по id товара удаляется товар, после чего коллекция перезаписывается, в зависимости от категории удаляемого товара, загружается коллекция с товарами конкретного продукта, после чего с ней осуществляются те же манипуляции, что и с коллекцией всех продуктов.

Следующим был разработан класс `FileManager`, ответственный за загрузку и запись в файл всех коллекций объектов различных классов, представленных ранее. Код класса `FileManager` частично представлен на рисунке 27.

Все поля данного класса хранят в себе информацию о пути к тому или иному файлу. Для того, чтобы программа запускалась и корректно работала, было принято решение записывать относительные, а не полные пути до файлов, в противном случае, из-за того, что невозможно отследить, по какому пути будет сохранён файл с программой, полный путь до файлов скорее всего будет отличаться.

Проанализировав рисунок, можно заметить, что методы класса разделены на две группы. В первой содержатся 4 «дженерик – метода», а во второй множественные их методы – обёртки, такой подход позволяет минимизировать повторение кода, а также уменьшить количество переменных, передаваемых в качестве параметров в метод.

```

import java.io.*;
import java.util.HashMap;
import java.util.Map;
import java.util.Scanner;
public class FileManager { 252 usages &#160; r1bn12 +1
    private static final String CLIENTFILE = "src/Clients.dat"; 2 usages
    private static final String SELLERSFILE = "src/Sellers.dat"; 2 usages
    private static final String PRODUCTFILE = "src/Products.dat"; 2 usages
    private static final String CPUTFILE = "src/FileCPU.dat"; 2 usages
    private static final String MOTHERBOARDFILE = "src/FileMotherBoard.dat"; 2 usages
    private static final String GPUFILE = "src/fileGPU.dat"; 2 usages
    private static final String RAMFILE = "src/FileRAM.dat"; 2 usages
    private static final String STORAGEFILE = "src/FileStorage.dat"; 2 usages
    private static final String POWERSUPPLYFILE = "src/FilePowerSupply.dat"; 2 usages
    private static final String CASEFILE = "src/FileCase.dat"; 2 usages
    private static final String COLLERFILE = "src/FileColler.dat"; 2 usages
    private static final String SOFTWAREFILE = "src/FileSoftware.dat"; 2 usages
    private static final String MONITORFILE = "src/FileMonitor.dat"; 2 usages
    private static final String MOUSEFILE = "src/FileMouse.dat"; 2 usages
    private static final String KEYBOARDFILE = "src/FileKeyboard.dat"; 2 usages

    public static <T extends User> void saveUser(Map<String, T> users, String filename){...}
    public static <T extends User> Map<String, T> loadUser(String filename){...}
    public static <T extends Product> void saveProduct(Map<String,T> products, String filename) 13 usages &#160; r1bn12
    {...}
    public static <T extends Product> Map<String, T> loadProduct(String filename) 13 usages &#160; r1bn12
    {...}
    //Классы обёртки
    public static void saveClient(Map<String,Client> clients) { saveUser(clients,CLIENTFILE); }
    public static void rewriteClient(Client client) {...}
    public static void saveSeller(Map<String,Seller> sellers) { saveUser(sellers,SELLERSFILE); }

    public static void saveProduct0(Map<String,Product> products) { saveProduct(products,PRODUCTFILE); }
    public static Map<String, Product> loadProduct0() { return loadProduct(PRODUCTFILE); }
    public static Map<String,Client> loadClient() { return loadUser(CLIENTFILE); }
    public static Map<String,Seller> loadSeller() { return loadUser(SELLERSFILE); }

    public static void saveCPU(Map<String,CPU> products) { saveProduct(products,CPUTFILE); }
    public static Map<String,CPU> loadCPU() { return loadProduct(CPUTFILE); }
    public static void saveMotherBoard(Map<String,MotherBoard> products) { saveProduct(products,MOTHERBOARDFILE); }
    public static Map<String, MotherBoard> loadMotherBoard() { return loadProduct(MOTHERBOARDFILE); }
    public static void saveGPU(Map<String,GPU> products) { saveProduct(products,GPUFILE); }
    public static Map<String, GPU> loadGPU() { return loadProduct(GPUFILE); }
    public static void saveRAM(Map<String, RAM> products) { saveProduct(products, RAMFILE); }
    public static Map<String, RAM> loadRAM() { return loadProduct(RAMFILE); }
    public static void saveStorage(Map<String,Storage> products) { saveProduct(products,STORAGEFILE); }
    public static Map<String, Storage> loadStorage() { return loadProduct(STORAGEFILE); }
    public static void savePowerSupply(Map<String,PowerSupply> products) { saveProduct(products,POWERSUPPLYFILE); }
    public static Map<String, PowerSupply> loadPowerSupply() { return loadProduct(POWERSUPPLYFILE); }
    public static void saveCase(Map<String,Case> products) { saveProduct(products,CASEFILE); }
    public static Map<String, Case> loadCase() { return loadProduct(CASEFILE); }
    public static void saveCooler(Map<String,Cooler> products) { saveProduct(products,COLLERFILE); }
    public static Map<String, Cooler> loadCooler() { return loadProduct(COLLERFILE); }
    public static void saveSoftware(Map<String,Software> products) { saveProduct(products,SOFTWAREFILE); }
    public static Map<String, Software> loadSoftware() { return loadProduct(SOFTWAREFILE); }
    public static void saveMonitor(Map<String,Monitor> products) { saveProduct(products,MONITORFILE); }
    public static Map<String, Monitor> loadMonitor() { return loadProduct(MONITORFILE); }
    public static void saveMouse(Map<String,Mouse> products) { saveProduct(products,MOUSEFILE); }
    public static Map<String,Mouse> loadMouse() { return loadProduct(MOUSEFILE); }
    public static void saveKeyboard(Map<String,Keyboard> products) { saveProduct(products,KEYBOARDFILE); }
    public static Map<String, Keyboard> loadKeyboard() { return loadProduct(KEYBOARDFILE); }
}

```

Рисунок 27 – Код класса FileManager

На рисунке 28 изображен код двух «дженерик – методов», saveProduct и loadProduct, отвечающих соответственно за загрузку из файла и сохранение в файл коллекций, несущих в себе объекты классов-наследников Product.

```
public static <T extends Product> void saveProduct(Map<String,T> products, String filename) 13 usages  rtbn12
{
    try {
        ObjectOutputStream ous = new ObjectOutputStream(new FileOutputStream(filename));
        ous.writeObject(products);
        ous.close();
    } catch (IOException e) {
        System.out.println("Ошибка чтения файла!");
    }
}

public static <T extends Product> Map<String, T> loadProduct(String filename) 13 usages  rtbn12
{
    try {
        ObjectInputStream ois = new ObjectInputStream(new FileInputStream(filename));
        Map<String,T> products = (Map<String,T>)ois.readObject();
        ois.close();
        return products;
    } catch (IOException | ClassNotFoundException e) {
        System.out.println("Ошибка загрузки: " + e.getMessage());
        System.out.println("Возвращаю пустую коллекцию!");
        return new HashMap<String, T>(); // ← return ВНУТРИ catch!
    }
}
```

Рисунок 28 – Код методов saveProduct и loadProduct

Для сохранения коллекции в файл её, а также путь до самого файла надо передать в качестве параметров для метода saveProduct. После чего вызывается блок «try-catch» в котором создаётся объект класса ObjectOutputStream, у которого вызывается метод writeObject, в который в качестве параметра передаётся сохраняемая коллекция, после чего ObjectOutputStream закрывается. В «catch» блоке происходит отработка ошибки чтения файла.

Схожая логика наблюдается и в методе loadProduct, только в нём в качестве параметра передаётся только путь до файла с коллекцией, а сама коллекция является возвращаемым элементом, вместо ObjectOutputStream создаётся и закрывается ObjectInputStream.

Классы обёртки просто конкретно задают пути до соответствующих файлов и тип возвращаемых элементов.

Завершающий и самый важный класс – ComputerShop, представляющий собой всю логику работы компьютерного магазина, в методе main реализована практически вся работа с консольным интерфейсом приложения. Сокращённый код ComputerShop представлен на рисунке 29, полный – в приложении А, рисунок 1.

```
import java.util.*;
import java.util.stream.Collectors;

import static java.util.Comparator.comparing;

public class ComputerShop {  ⚡ rtbn12+1
    private static final String address = "Belgorod, Pobedy street, 85b"; 1 usage
    private static final String name = "ENS"; 2 usages
    private static final String phone = "+79024292753"; 1 usage

    private static Client client = null; 23 usages
    private static Seller seller = null; 8 usages

    public static void main(String[] args) {...}

    public static void aboutShop(){...}

    public static String getName() { return name; }

    private static void handleProductMenu(String categoryName, Scanner scanner) {...}

    private static void productActionsMenu( Scanner scanner) {...}

    public static void handleProductSellerMenu( Scanner scanner){...}

    public static void Migrator(){...}
}
```

Рисунок 29 – Код класса ComputerShop

Поля client и seller при запуске программы принимают пустое значение, так как они ссылочного типа, то просто изначально ни на что не ссылаются. После завершения процесса авторизации клиента или продавца, они ссылаются соответственно на эти объекты. На рисунке 30 приведён код метода aboutShop, ответственного за вывод информации о компьютерном магазине.

```

public static void aboutShop() { 1 usage  rtbn12
    System.out.println("\nНазвание магазина: " + name);
    System.out.println("Адрес: " + address);
    System.out.println("Контактный телефон: " + phone);
    System.out.println("\nДанный компьютерный магазин не существует в реальности, \n" +
        "а представляет собой лишь некоммерческую программу, \n" +
        "написанную студентом в качестве курсовой работы, \n" +
        "все совпадения с возможной логикой работы магазина 'DNS' вовсе не случайны, \n" +
        "так как лучший способ создать что-то хорошее - брать пример с лучших!");
}

```

Рисунок 30 – Код метода aboutShop

Методы `handleProductMenu`, `productActionsMenu` и `handleProductSellerMenu`, являются определёнными частями кода, который должен несколько раз в неизменном виде повторяться в разных частях метода `main`.

В процессе разработки программы неоднократно возникали случаи, когда приходилось менять «сеттеры» для классов продуктов, в следствие чего возникала проблема не сходимости версий сериализации. Если изменения были минимальными – помогал специальный метод `Migrator`, его код частично представлен на рисунке 31.

```

public static void Migrator() { no usages  Egor Solodov
    System.out.println("=== Начало миграции товаров ===");

    // 1. Процессоры
    Map<String, CPU> cpuMap = FileManager.loadCPU();
    FileManager.saveCPU(cpuMap);
    System.out.println("CPU мигрированы: " + cpuMap.size());

    // 2. RAM
    Map<String, RAM> ramMap = FileManager.loadRAM();
    FileManager.saveRAM(ramMap);
    System.out.println("RAM мигрированы: " + ramMap.size());

    // 3. Материнские платы
    Map<String, MotherBoard> mbMap = FileManager.loadMotherBoard();
    FileManager.saveMotherBoard(mbMap);
    System.out.println("MotherBoard мигрированы: " + mbMap.size());
}

```

Рисунок 31 – Код метода Migrator

Его принцип заключается в последовательной загрузке коллекций разных категорий товаров и их перезаписи. Но метод не всемогущ, при малейшем изменении структуры поля класса или его удалении, или добавлении нового – метод не сработает.

Очень сокращённый код метода main представлен на рисунке 32.

```
public static void main(String[] args) { // rtbn12 +1
    Scanner scanner = new Scanner(System.in);

    / Migrator();// применяется при изменении методов классов

    boolean cycle = true;
    while (cycle)
    {
        System.out.println("\n\n+++++Компьютерный магазин+++++");
        System.out.print("Введите номер интересующего раздела:\n" +
            "1 - Покупателю\n" +
            "2 - Продавцу\n" +
            "3 - Подробнее о магазине\n" +
            "0 - Выход\n" +
            "Ваш выбор:");

        int choice;

        try {
            choice = scanner.nextInt();
            switch (choice)
            {
                ...
            }
        } catch (InputMismatchException e) {
            System.out.println("Произошла ошибка!\n" +
                "Пожалуйста, введите корректное целочисленное значение выбранного варианта!\n" +
                "В прошлый раз вы ввели букву вместо числа!");
            scanner.nextLine();
        }
        catch (Exception a) {
            System.out.println("Произошла неизвестная ошибка!");
            scanner.nextLine();
        }
    }

    scanner.close();
}
```

Рисунок 32 – Код метода main

Фактически, весь метод состоит главного меню и его подменю, управление построено путём ввода соответствующих цифр, обозначающих конкретное действие в меню, для предотвращения любых ошибок ввода на любом уровне меню используются «try – catch» блоки.

В этом разделе были описаны все реализованные классы и их методы. Для ознакомления с полным кодом проекта в приложении А, рисунок 2 приведены ссылки для копирования проекта с открытого репозитория «GitHub».

2.3 Демонстрация работы программы

При запуске программы пользователя в консоли встречает главное меню программы, изображенное на рисунке 33.

```
+---+---+---+---+---Компьютерный магазин---+---+---+---+---+
Введите номер интересующего раздела:
1 - Покупателю
2 - Продавцу
3 - Подробнее о магазине
0 - Выход
Ваш выбор:
```

Рисунок 33 – Главное меню программы

При вводе цифры 3 в консоли появляется информация об компьютерном магазине, при вводе 0 – программа прекращает свою работу (программа не прекратит свою работу, пока в данном меню пользователь не введёт 0), при вводе буквы или цифры, не входящей в перечень доступных вариантов – появляется сообщение об ошибке, но программа продолжает свою работу. Данные действия продемонстрированы на рисунке 34 и 35.

```
+---+---+---+---+---Компьютерный магазин---+---+---+---+---+
Введите номер интересующего раздела:
1 - Покупателю
2 - Продавцу
3 - Подробнее о магазине
0 - Выход
Ваш выбор:3

Название магазина:ENS
Адрес:Belgorod, Pobedy street, 85b
Контактный телефон:+79024292753
```

Данный компьютерный магазин не существует в реальности,
а представляет собой лишь некоммерческую программу,
написанную студентом в качестве курсовой работы,
все совпадения с возможной логикой работы магазина 'DNS' вовсе не случайны,
так как лучший способ создать что-то хорошее - брать пример с лучших!

Рисунок 34 – Вывод информации об компьютерном магазине

```

+---+---+---+---+---Компьютерный магазин---+---+---+---+---+
Введите номер интересующего раздела:
1 - Покупателю
2 - Продавцу
3 - Подробнее о магазине
0 - Выход
Ваш выбор:c
Произошла ошибка!
Пожалуйста, введите корректное целочисленное значение выбранного варианта!
В прошлый раз вы ввели букву вместо числа!

```

```

+---+---+---+---+---Компьютерный магазин---+---+---+---+---+
Введите номер интересующего раздела:
1 - Покупателю
2 - Продавцу
3 - Подробнее о магазине
0 - Выход
Ваш выбор:55
Такого варианта выбора нет!
Пожалуйста, введите корректную цифру!

```

```

+---+---+---+---+---Компьютерный магазин---+---+---+---+---+
Введите номер интересующего раздела:
1 - Покупателю
2 - Продавцу
3 - Подробнее о магазине
0 - Выход
Ваш выбор:0
Выражаю благодарность за пользование программой!
До новых встреч!

```

```

Process finished with exit code 0

```

Рисунок 35 – Обработка ошибок некорректного ввода и выход из программы

Далее в зависимости от того, кто на данный момент пользуется программой, покупатель или продавец следует ввести на клавиатуре соответственно цифру 1 или 2.

При вводе 1 в консоли появляется раздел покупателя, изображённый на рисунке 36.


```
+--+--+--+--+--+--Раздел покупателя--+--+--+--+--+--+  
Введите номер интересующего действия:  
1 - Регистрация  
2 - Авторизация  
3 - Каталог товаров  
4 - Поиск товара по id  
5 - Проверить баланс  
6 - Корзина  
7 - Список покупок  
0 - Выход в главное меню  
Ваш выбор:
```

Рисунок 36 – Раздел покупателя

Тут клиенту открывается широкий спектр возможностей, реализованный в программе. При вводе 1 начинается процесс регистрации, изображённый на рисунке 37.

```
Приветствую Вас, новый пользователь!  
Для успешного прохождения процесса регистрации вам предстоит ввести своё НАСТОЯЩЕЕ имя,  
а также, придумать логин и пароль!  
Значение пароля при первичном и вторичном вводе должны быть идентичны!  
Введите ваше ИМЯ:Egor  
  
Придумайте и введите ваш ЛОГИН:egor0  
  
Такой логин уже занят!  
Пожалуйста, придумайте и введите другой:egor1  
  
Придумайте и введите ваш ПАРОЛЬ:  
123  
  
Введите пароль ещё раз:  
1234  
  
Вторичный пароль не соответствует первичному!  
Попробуйте снова!:  
Введите пароль ещё раз:  
123  
  
Пароли совпали!  
Регистрация прошла успешно!  
Спасибо за регистрацию!
```

Рисунок 37 – Процесс регистрации нового покупателя

При вводе 2 запускается процесс авторизации покупателя, изображённый на рисунке 38.

```
С возвращением, дорогой покупатель!
Сейчас вам предстоит ввести ваши ранее придуманные логин и пароль!
Введите ваш логин:egor55

Покупатель с таким логином пока не зарегистрирован!
Пожалуйста, введите корректный логин:egor1

Введите ваш пароль:12345

Вы ввели неверный пароль!
Пожалуйста, попробуйте ввести его ещё раз:123

Поздравляю с успешной авторизацией!
Доброго времени суток, Егор!
```

Рисунок 38 – Процесс авторизации покупателя

При выборе варианта 3 в консоли покупателю выводится весь каталог доступных категорий товаров, изображённый на рисунке 39.

```
+-----Каталог товаров-----+

Выберите интересующую категорию товара:
1 - Центральный процессор
2 - Материнская плата
3 - Видеокарта
4 - Оперативная память
5 - Постоянное запоминающее устройство
6 - Блок питания
7 - Корпус
8 - Кулер
9 - Программное обеспечение
10 - Монитор
11 - Мышь
12 - Клавиатура
0 - Выход в меню
Ваш выбор:
```

Рисунок 39 – Каталог товаров компьютерного магазина

Далее покупатель либо вводит номер интересующего его раздела, либо возвращается в раздел покупателя, в качестве примера на рисунке 40 изображено состояние консоли при вводе варианта 3.

```

Ваш выбор:3
===== Базовая информация =====

Id: GPU1
Категория: Видеокарта
Бренд: Palit
Модель: GeForce RTX 5070 Infinity 3
Цена: 63500
Рейтинг: 4.7
Количество на складе: 3 шт.
=====

Id: GPU0
Категория: Видеокарта
Бренд: MSI
Модель: GeForce RTX 5060 VENTUS 2X OC
Цена: 37000
Рейтинг: 5.0
Количество на складе: 5 шт.
=====

Id: GPU2
Категория: Видеокарта
Бренд: Palit
Модель: GeForce RTX 5080 GamingPro
Цена: 130000
Рейтинг: 4.82
Количество на складе: 1 шт.
=====

Выберите одно из доступных действий:
1 - Подробнее о товаре
2 - Вывести товары только в выделенном диапазоне
3 - Вывести товары только ниже введённой цены
4 - Вывести товары только выше указанной цены
5 - Сортировать по увеличению цены
6 - Сортировать по уменьшению цены
7 - Сортировать по уменьшению рейтинга
8 - Сортировать по увеличению рейтинга
9 - Сбросить сортировки
0 - Вернуться к каталогу товаров
Ваш выбор:

```

Рисунок 40 – Каталог видеокарт

На данном этапе клиент в зависимости от введенной цифры может подробнее узнать о заинтересовавшем его товаре, применить фильтрацию или сортировку, вернуться к каталогу товаров. На рисунке 41 показана работа фильтра, после которого в консоль выводятся только товары в определенном диапазоне, на рисунке 42 – работа сортировки по уменьшению рейтинга.

```
Ваш выбор: 2

Введите нижний порог интересующей цены:c
Произошла ошибка!
Пожалуйста, введите корректное целочисленное значение выбранного варианта!
В прошлый раз вы ввели букву вместо числа!

Введите нижний порог интересующей цены:100

Введите верхний порог интересующей цены:1000

Не найдено товаров, удовлетворяющих критериям вашего отбора!

Выберите одно из доступных действий:
1 - Подробнее о товаре
2 - Вывести товары только в выделенном диапазоне
3 - Вывести товары только ниже введенной цены
4 - Вывести товары только выше указанной цены
5 - Сортировать по увеличению цены
6 - Сортировать по уменьшению цены
7 - Сортировать по уменьшению рейтинга
8 - Сортировать по увеличению рейтинга
9 - Сбросить сортировки
0 - Вернуться к каталогу товаров
Ваш выбор: 2

Введите нижний порог интересующей цены:30000

Введите верхний порог интересующей цены:40000

==+==+==+==+==+Товары в ценовом диапазоне от 30000,00 до 40000,00 рублей==+==+==+==+==+

Id: GPU0
Категория: Видеокарта
Бренд: MSI
Модель: GeForce RTX 5060 VENTUS 2X OC
Цена: 37000
Рейтинг: 5.0
Количество на складе: 5 шт.
=====

Выберите одно из доступных действий:
1 - Подробнее о товаре
2 - Вывести товары только в выделенном диапазоне
3 - Вывести товары только ниже введенной цены
4 - Вывести товары только выше указанной цены
5 - Сортировать по увеличению цены
6 - Сортировать по уменьшению цены
7 - Сортировать по уменьшению рейтинга
8 - Сортировать по увеличению рейтинга
9 - Сбросить сортировки
0 - Вернуться к каталогу товаров
```

Рисунок 41 – Работа фильтра по ценовому диапазону

```

Ваш выбор: 7

=====Товары, отсортированные по уменьшению рейтинга=====

Id: GPU0
Категория: Видеокарта
Бренд: MSI
Модель: GeForce RTX 5060 VENTUS 2X OC
Цена: 37000
Рейтинг: 5.0
Количество на складе: 5 шт.
=====

Id: GPU2
Категория: Видеокарта
Бренд: Palit
Модель: GeForce RTX 5080 GamingPro
Цена: 130000
Рейтинг: 4.82
Количество на складе: 1 шт.
=====

Id: GPU1
Категория: Видеокарта
Бренд: Palit
Модель: GeForce RTX 5070 Infinity 3
Цена: 63500
Рейтинг: 4.7
Количество на складе: 3 шт.
=====

Выберите одно из доступных действий:
1 - Подробнее о товаре
2 - Вывести товары только в выделенном диапазоне
3 - Вывести товары только ниже введённой цены
4 - Вывести товары только выше указанной цены
5 - Сортировать по увеличению цены
6 - Сортировать по уменьшению цены
7 - Сортировать по уменьшению рейтинга
8 - Сортировать по увеличению рейтинга
9 - Сбросить сортировки
0 - Вернуться к каталогу товаров
Ваш выбор: 9

```

Рисунок 42 – Работа сортировки по уменьшению рейтинга

При вводе 1 клиенту предстоит правильно ввести id заинтересовавшего его товара, после чего перед ним в консоли появится полная информация о данном продукте и меню выбора следующих действий. Состояние консоли при вводе 1 в каталоге видеокарт показано на рисунке 43.

Ваш выбор: 1

Введите id заинтересовавшего вас товара: GPU66

Товара с таким id не найдено!

Пожалуйста, попробуйте ввести id товара ещё раз: GPU1

Id: GPU1
Категория: Видеокарта
Бренд: Palit
Модель: GeForce RTX 5070 Infinity 3
Цена: 63500 рублей
Интерфейсы подключения задней панели: PCIe
Объем видеопамати: 12 Гб GDDR7
Количество вычислительных ядер: 6144
Базовая частота: 2325.0 МГц
Турбо частота: 2510.0 МГц
Ширина шины памяти: 192 бит
Тип охлаждения: воздушное
Разъемы питания: 2x 8-pin
Поддерживаемые видеовыходы: HDMI: 1, DisplayPort: 3, VGA: 0, DVI-D: 0, DVI-I: 0
Версия PCIe: PCIe 5.0
Потребляемая мощность: 250 Вт
Рабочее напряжение: 1.0 V
Страна-производитель: Китай
Год релиза: 2025
Гарантийный срок: 36 месяцев
Рейтинг: 4.7 из 5!
Количество на складе: 3 шт.
Описание: Видеокарта повышенной производительности, легко справится с любой задачей

Выберите одно из доступных действий:

- 1 - Купить товар
- 2 - Положить товар в корзину
- 3 - Авторизоваться
- 0 - Назад

Ваш выбор:

Рисунок 43 – Полное описание товара и меню следующих действий

После чего, в зависимости от введенной цифры, клиент может либо купить данный товар, либо положить его в свою корзину, данные действия не будут работать, пока пользователь не авторизован, поэтому, чтобы не заставлять покупателя возвращаться в раздел покупателя и авторизовываться, данное действие доступно и на этом этапе, вариант 0 возвращает пользователя обратно в каталог видеокарт. Процесс покупки товара изображён на рисунке 44, на рисунке 45 – процесс добавления товара в корзину, следует заметить, что после покупки товара, количество его единиц в каталоге уменьшится,

чтобы продемонстрировать, что процесс авторизации на данном этапе работает, программа была запущена заново и пользователь в начале не авторизовывался.

```
Ваш выбор: 2

Для выполнения этого действия сначала надо пройти процесс авторизации!

Выберите одно из доступных действий:
1 - Купить товар
2 - Положить товар в корзину
3 - Авторизоваться
0 - Назад
Ваш выбор: 3
С возвращением, дорогой покупатель!
Сейчас вам предстоит ввести ваши ранее придуманные логин и пароль!
Введите ваш логин:egor1

Введите ваш пароль:123

Поздравляю с успешной авторизацией!
Доброго времени суток, Егор!

Выберите одно из доступных действий:
1 - Купить товар
2 - Положить товар в корзину
3 - Авторизоваться
0 - Назад
Ваш выбор: 1

На вашем балансе недостаточно средств!
Ваш текущий баланс составляет: 0.0 рублей(ля)!

Для продолжения процесса покупки необходимо пополнить баланс минимум на 63500.0 рублей(ля)!
Выберите одно из доступных действий:
1 - пополнить баланс на 63500.0 рублей(ля)
0 - Прервать процесс покупки
Ваш выбор:1

Вы подтверждаете покупку товара?
(Подтверждаете, что совершаете покупку товара осознано, без внешнего принуждающего воздействия)
В случае успешной покупки, купленный вами товар отобразится в вашем списке покупок,
а с вашего счёта спишется 63500 рублей(ля)
Выберите одно из доступных действий:
1 - Подтвердить и завершить процесс покупки
0 - Прервать процесс покупки
Ваш выбор:1

Покупка товара прошла успешно!

Выберите одно из доступных действий:
1 - Купить товар
2 - Положить товар в корзину
3 - Авторизоваться
0 - Назад
Ваш выбор: |
```

Рисунок 44 – Процесс покупки товара

Количество на складе: 2 шт.

Описание: Видеокарта повышенной производительности, легко справится с любой задачей

Выберите одно из доступных действий:

- 1 - Купить товар
- 2 - Положить товар в корзину
- 3 - Авторизоваться
- 0 - Назад

Ваш выбор: 2

Для выполнения этого действия сначала надо пройти процесс авторизации!

Выберите одно из доступных действий:

- 1 - Купить товар
- 2 - Положить товар в корзину
- 3 - Авторизоваться
- 0 - Назад

Ваш выбор: 3

С возвращением, дорогой покупатель!

Сейчас вам предстоит ввести ваши ранее придуманные логин и пароль!

Введите ваш логин: `egor1`

Введите ваш пароль: `123`

Поздравляю с успешной авторизацией!

Доброго времени суток, Егор!

Выберите одно из доступных действий:

- 1 - Купить товар
- 2 - Положить товар в корзину
- 3 - Авторизоваться
- 0 - Назад

Ваш выбор: 2

Товар успешно поступил в вашу корзину!

Выберите одно из доступных действий:

- 1 - Купить товар
- 2 - Положить товар в корзину
- 3 - Авторизоваться
- 0 - Назад

Ваш выбор: |

Рисунок 45 – Процесс добавления товара в корзину

На этом функционал раздела «Каталог товаров» окончен, при возвращении в раздел покупателя клиент так же ввести вариант 4 и выполнить поиск товара по id после чего в консоли отобразится уже знакомый интерфейс, содержание которого показано на рисунке 46.


```

Введите номер интересующего действия:
1 - Регистрация
2 - Авторизация
3 - Каталог товаров
4 - Поиск товара по id
5 - Проверить баланс
6 - Корзина
7 - Список покупок
0 - Выход в главное меню
Ваш выбор:4

+--+--+--+--+Раздел поиска товара по id--+--+--+--+

Выберите одно из доступных действий:
1 - Выполнить поиск
0 - Выход в меню покупателя
Ваш выбор:1

Введите id интересовавшего вас товара:444

Товара с таким id не найдено!
Пожалуйста, попробуйте ввести id товара ещё раз:RAM0

Id: RAM0
Категория: Оперативная память
Бренд: Kingston
Модель: KingstonFURY Beast Black
Цена: 15000 рублей
Интерфейс подключения: DIMM
Объем одного модуля: 8 Гб
Количество модулей: 2
Общий объем комплекта: 16 Гб
Частота: 3200 МГц
Тайминги: CL14
Тип памяти: DDR4
RGB подсветка: Нет
Радиатор охлаждения: Да
Потребляемая мощность: 5 Вт
Рабочее напряжение: 1.35 V
Страна-производитель: Китай
Год релиза: 2024
Гарантийный срок: 120 месяцев
Рейтинг: 4.4из 5!
Количество на складе: 2 шт.
Описание: Оперативная память Kingston FURY Beast Black способна обеспечить высокие показатели производительности и скорости в составе игрового компьютера.

Выберите одно из доступных действий:
1 - Купить товар
2 - Положить товар в корзину
3 - Авторизоваться
0 - Назад
Ваш выбор:

```

Рисунок 46 – Процесс поиска товара по id

Чтобы проверить свой баланс, клиенту необходимо ввести цифру 5, чтобы просмотреть содержимое корзины – 6, списка совершённых покупок – 7, для выполнения этих действий, покупатель обязательно должен быть авторизован. Состояние консоли при переходе в раздел баланса изображено на рисунке 47, при переходе в раздел корзины – 48, при переходе в список совершённых покупок – 49.

```

+-----+-----Раздел покупателя-----+
Введите номер интересующего действия:
1 - Регистрация
2 - Авторизация
3 - Каталог товаров
4 - Поиск товара по id
5 - Проверить баланс
6 - Корзина
7 - Список покупок
0 - Выход в главное меню
Ваш выбор:5

+-----+-----Баланс-----+
Ваш баланс составляет: 0.0 рублей!

Выберите одно из доступных действий:
1 - Пополнить баланс
0 - Выход в меню покупателя
Ваш выбор:1

Выберите вносимую сумму в рублях:1234

+-----+-----Баланс-----+
Ваш баланс составляет: 1234.0 рублей!

Выберите одно из доступных действий:
1 - Пополнить баланс
0 - Выход в меню покупателя
Ваш выбор:0

```

Рисунок 47 — Раздел баланса пользователя

```

+-----+-----Раздел покупателя-----+
Введите номер интересующего действия:
1 - Регистрация
2 - Авторизация
3 - Каталог товаров
4 - Поиск товара по id
5 - Проверить баланс
6 - Корзина
7 - Список покупок
0 - Выход в главное меню
Ваш выбор:6

+-----+-----Ваша корзина-----+

Корзина обновлена!

Id: GPU1
Категория: Видеокарта
Бренд: Palit
Модель: GeForce RTX 5070 Infinity 3
Цена: 63500
Рейтинг: 4.7
Количество на складе: 2 шт.

=====

Выберите одно из доступных действий:
1 - Подробнее о каком-либо товаре
2 - Купить все товары
3 - Купить товар по его id
4 - Очистить корзину
5 - Удалить товар из корзины по его id
0 - Выход в меню покупателя
Ваш выбор:

```

Рисунок 48 — Раздел корзины покупателя

```

+-+--+--+--+--+Раздел покупателя--+-+--+--+--+
Введите номер интересующего действия:
1 - Регистрация
2 - Авторизация
3 - Каталог товаров
4 - Поиск товара по id
5 - Проверить баланс
6 - Корзина
7 - Список покупок
0 - Выход в главное меню
Ваш выбор:7

+-+--+--+--+--+Ваш список покупок--+-+--+--+--+

Id: GPU1
Категория: Видеокарта
Бренд: Palit
Модель: GeForce RTX 5070 Infinity 3
Цена: 63500
Рейтинг: 4.7
Количество на складе: 2 шт.
Дата покупки товара: Thu Jan 08 22:57:58 MSK 2026

=====

Выберите одно из доступных действий:
1 - Подробнее о товаре товаре:
0 - Выход в меню покупателя
Ваш выбор:

```

Рисунок 49 – Список совершённых покупок

На этом возможности покупателя как пользователя оканчиваются.

При выборе в главном меню программы варианта 2 осуществляется переход в раздел продавца, изображённого на рисунке 50.

```

+-+--+--+--+--+Компьютерный магазин--+-+--+--+--+
Введите номер интересующего раздела:
1 - Покупателю
2 - Продавцу
3 - Подробнее о магазине
0 - Выход
Ваш выбор:2

+-+--+--+--+--+Раздел продавца--+-+--+--+--+
Введите номер интересующего действия:
1 - Регистрация
2 - Авторизация
3 - Добавить товар
4 - Каталог товаров
5 - Поиск товара по id
6 - Изменить товар
7 - Удалить товар
0 - Выход в главное меню
Ваш выбор:

```

Рисунок 50 – Раздел продавца

Так же, как и покупателю, в программе «Компьютерный магазин» продавцу доступно 8 вариантов действий. Авторизация продавца практически ничем не отличается от клиентской, за исключением других приветственных слов. А вот процесс регистрации уже имеет одну важную особенность – ввод кодового слова продавца, он изображён на рисунке 51.

```
Ваш выбор:1

Приветствуем вас в компании 'ENS' !
Мы ценим каждого сотрудника и очень рады от того, что вы решили присоединиться к нашей дружной семье!
Сейчас вам предстоит Ввести ваше НАСТОЯЩЕЕ имя, а также придумать и ввести логин и пароль,
а также специальное кодовое слово (вам его сообщил действующий сотрудник)!
Для успешного прохождения регистрации значение пароля при первичном и вторичном вводе должны быть идентичны!
Введите ваше ИМЯ:Egor

Придумайте и введите ваш ЛОГИН:egor0

Такой логин уже занят!
Пожалуйста, придумайте и введите другой:egor1

Придумайте и введите ваш ПАРОЛЬ:123

Введите пароль ещё раз:124

Первичный пароль не соответствует вторичному!
Попробуйте снова:123

Пароли совпали!
Введите кодовое слово:BSTU

Вы ввели неверное кодовое слово!
Попробуйте ещё раз!:BSU

Регистрация прошла успешно!
Спасибо за то, что выбрали работать в нашей компании!
```

Рисунок 51 – Процесс регистрации нового продавца

Чтобы продавцу добавить новый продукт в каталог товаров, необходимо ввести в консоль цифру 3, чтобы попасть в каталог товаров – 4, чтобы выполнить поиск товара по id – 5, чтобы изменить товар – 6, чтобы удалить товар – 7, чтобы завершить работу программы – 0. Чтобы работали все действия из перечисленных (кроме последнего), продавец должен быть авторизован. На рисунках 52 и 53 изображён процесс добавления нового продукта в каталог товаров.

Ваш выбор: 3

Выберите категорию товара, который хотите добавить:

- 1-Центральный процессор
 - 2-Материнская плата
 - 3-Видеокарта
 - 4-Оперативная память
 - 5-Постоянное запоминающее устройство
 - 6-Блок питания
 - 7-Корпус
 - 8-Кулер
 - 9-Программное обеспечение
 - 10-Монитор
 - 11-Мышь
 - 12-Клавиатура
 - 0-Выход в меню продавца
- Ваш выбор: 9

=== Регистрация программного обеспечения ===

Введите ID продукта: SW2

Такой ID уже занят!

Придумайте другой: SW3

Введите бренд: Nintendo

Введите модель: The Legend of Zelda

Введите страну производителя: Япония

Введите год производства: 2023

Введите цену товара: 6100

Введите описание товара: популярная игра в жанре приключенческого файтинга

Введите срок предоставляемой гарантии на товар в целом количестве месяцев: 6

Произошла ошибка!

Пожалуйста, введите корректное целочисленное значение срока гарантии!

В прошлый раз вы ввели букву вместо числа!

Введите срок предоставляемой гарантии на товар в целом количестве месяцев: 0

Введите количество единиц товара, поступающего на склад: 3

Введите рейтинг товара (0-5): 4,76

Рисунок 52 – Начало процесса добавления нового товара

Введите рейтинг товара (0-5): 4,76

Выберите тип программного обеспечения:

- 1 - Операционная система
- 2 - Антивирус
- 3 - Офисный пакет
- 4 - Графический редактор
- 5 - Игра
- 6 - Утилита
- 7 - Драйвер

Ваш выбор: 5

Введите срок лицензии в месяцах (0 = бессрочная, 1-120 месяцев): 0

Выберите поддерживаемые операционные системы (можно выбрать несколько цифр через пробел):

- 1 - Windows 10/11
- 2 - Windows 7/8
- 3 - macOS
- 4 - Linux
- 5 - Android
- 6 - iOS
- 7 - PS5
- 8 - PS4/5
- 9 - XBOX
- 10 - Nintendo Switch

Введите номера выбранных ОС (например: 1 3 4): 10

Введите системные требования (можно подробно описать требования к железу): Nintendo Switch

Выберите тип версии:

- 1 - Цифровая версия (электронный ключ/файл)
- 2 - Коробочная версия (физический носитель)

Ваш выбор: 2

Выберите поддерживаемые языки (можно выбрать несколько цифр через пробел):

- 1 - Русский
- 2 - Английский
- 3 - Немецкий
- 4 - Французский
- 5 - Испанский
- 6 - Китайский
- 7 - Японский
- 8 - Многоязычная версия

Введите номера выбранных языков (например: 1 2 8): 1 2

Программное обеспечение Nintendo The Legend of Zelda успешно зарегистрировано!

Рисунок 53 – Завершение процесса добавления нового товара

На рисунках 54-55 изображён каталог товаров для продавца и доступные в нём действия.

Ваш выбор: 9

===== Каталог ПО =====

Id: SW1
Категория: Программное обеспечение
Бренд: Nintendo
Модель: Mario Kart World
Тип программного обеспечения: Игра
Срок лицензирования: 0 месяцев
Поддерживаемые операционные системы: Nintendo Switch
Системные требования: Nintendo Switch
Цифровая версия: Нет
Поддерживаемые языки: Русский, Английский
Цена: 9000 рублей
Страна-производитель: Япония
Год релиза: 2025
Гарантийный срок: 0 месяцев
Рейтинг: 5.0из 5!
Количество на складе: 4 шт.
Описание: Отправляйтесь в путь вместе с Марио и его друзьями! Гоняйте по тр
Дата добавления товара: Mon Dec 29 22:06:20 MSK 2025

=====

Id: SW0
Категория: Программное обеспечение
Бренд: Microsoft
Модель: MicrosoftWindows 11 Pro
Тип программного обеспечения: Операционная система
Срок лицензирования: 0 месяцев
Поддерживаемые операционные системы: Windows 10/11
Системные требования: Процессор: не менее 2 ядер, частота от 1 ГГц, ОЗУ: 4
Цифровая версия: Да
Поддерживаемые языки: Многоязычная
Цена: 7900 рублей
Страна-производитель: США
Год релиза: 2021
Гарантийный срок: 0 месяцев
Рейтинг: 4.44из 5!
Количество на складе: 2 шт.
Описание: Windows 11 Pro профессиональная версия популярной операционной си
Дата добавления товара: Mon Dec 29 22:01:18 MSK 2025

=====

Рисунок 54 – Начало каталога товаров продавца

Id: SW3
Категория: Программное обеспечение
Бренд: Nintendo
Модель: The Legend of Zelda
Тип программного обеспечения: Игра
Срок лицензирования: 0 месяцев
Поддерживаемые операционные системы: Nintendo Switch
Системные требования: Nintendo Switch
Цифровая версия: Нет
Поддерживаемые языки: Русский, Английский
Цена: 6100 рублей
Страна-производитель: Япония
Год релиза: 2023
Гарантийный срок: 0 месяцев
Рейтинг: 4.76из 5!
Количество на складе: 3 шт.
Описание: популярная игра в жанре приключенческого файтинга
Дата добавления товара: Thu Jan 08 23:54:26 MSK 2026

=====

Id: SW2
Категория: Программное обеспечение
Бренд: Rockstar
Модель: Grand Theft Auto V
Тип программного обеспечения: Игра
Срок лицензирования: 0 месяцев
Поддерживаемые операционные системы: PS5
Системные требования: PS5
Цифровая версия: Нет
Поддерживаемые языки: Русский, Английский
Цена: 2950 рублей
Страна-производитель: Австрия
Год релиза: 2022
Гарантийный срок: 0 месяцев
Рейтинг: 4.89из 5!
Количество на складе: 4 шт.
Описание: Лос-Сантос - город солнца, старлеток и вышедших в тираж звезд. Неког
Дата добавления товара: Sun Jan 04 14:31:40 MSK 2026

=====

Выберите одно из доступных действий:
1 - Добавить товар
2 - Изменить товар
3 - Удалить товар
0 - Вернуться к каталогу товаров
Ваш выбор:

Рисунок 55 – Каталог программного обеспечения и доступные дальнейшие действия

На рисунке 56 изображено состояние консоли при переходе в раздел поиска товара по id.

```
+--+--+--+--+--+Раздел продавца--+--+--+--+--+
Введите номер интересующего действия:
1 - Регистрация
2 - Авторизация
3 - Добавить товар
4 - Каталог товаров
5 - Поиск товара по id
6 - Изменить товар
7 - Удалить товар
0 - Выход в главное меню
Ваш выбор:5
===== Поиск товара по ID =====

Введите id заинтересовавшего вас товара:SW450

Товара с таким id не найдено!
Пожалуйста, попробуйте ввести id товара ещё раз:SW3

Id: SW3
Категория: Программное обеспечение
Бренд: Nintendo
Модель: The Legend of Zelda
Тип программного обеспечения: Игра
Срок лицензирования: 0 месяцев
Поддерживаемые операционные системы: Nintendo Switch
Системные требования: Nintendo Switch
Цифровая версия: Нет
Поддерживаемые языки: Русский, Английский
Цена: 6100 рублей
Страна-производитель: Япония
Год релиза: 2023
Гарантийный срок: 0 месяцев
Рейтинг: 4.76из 5!
Количество на складе: 3 шт.
Описание: популярная игра в жанре приключенческого файтинга
Дата добавления товара: Thu Jan 08 23:54:26 MSK 2026

Введите 0 для возвращения к меню продавца:
Введите 1 - чтобы повторно выполнить поиск:
Ваш выбор:
```

Рисунок 56 – Поиск товара по id

На рисунках 57 – 61 изображён процесс редактирования продукта из каталога товаров.

```

+-+--+--+--+--+Раздел продавца--+-+--+--+--+
Введите номер интересующего действия:
1 - Регистрация
2 - Авторизация
3 - Добавить товар
4 - Каталог товаров
5 - Поиск товара по id
6 - Изменить товар
7 - Удалить товар
0 - Выход в главное меню
Ваш выбор:6

+++++ Изменение товара +++++

Выберите одно из доступных действий:
1 - Точно изменить товар
0 - Вернуться назад
Ваш выбор:1

Введите id заинтересовавшего вас товара:SW3

Id: SW3
Категория: Программное обеспечение
Бренд: Nintendo
Модель: The Legend of Zelda
Тип программного обеспечения: Игра
Срок лицензирования: 0 месяцев
Поддерживаемые операционные системы: Nintendo Switch
Системные требования: Nintendo Switch
Цифровая версия: Нет
Поддерживаемые языки: Русский, Английский
Цена: 6100 рублей
Страна-производитель: Япония
Год релиза: 2023
Гарантийный срок: 0 месяцев
Рейтинг: 4.76из 5!
Количество на складе: 3 шт.
Описание: популярная игра в жанре приключенческого файтинга
Дата добавления товара: Thu Jan 08 23:54:26 MSK 2026

Вы точно хотите продолжить процесс редактирования данного товара товара?
Введите 1 - если хотите продолжить процесс редактирования товара!
Введите 0 - если хотите прервать процесс редактирования!
Ваш выбор:1

```

Рисунок 57 – Начало процесса редактирования товара

+--+--+--+--+--Редактирование программного обеспечения (Nintendo The Legend of Zelda ID: SW3)--+--+--+--+--+

Id: SW3
Категория: Программное обеспечение
Бренд: Nintendo
Модель: The Legend of Zelda
Тип программного обеспечения: Игра
Срок лицензирования: 0 месяцев
Поддерживаемые операционные системы: Nintendo Switch
Системные требования: Nintendo Switch
Цифровая версия: Нет
Поддерживаемые языки: Русский, Английский
Цена: 6100 рублей
Страна-производитель: Япония
Год релиза: 2023
Гарантийный срок: 0 месяцев
Рейтинг: 4.76из 5!
Количество на складе: 3 шт.
Описание: популярная игра в жанре приключенческого файтинга

Выберите номер поля, которое собрались редактировать:
(Или введите 0, если вы уже отредактировали всё, что нужно):

- 1 - Цена
 - 2 - Тип программного обеспечения
 - 3 - Срок лицензии
 - 4 - Поддерживаемые операционные системы
 - 5 - Системные требования
 - 6 - Тип версии (цифровая/коробочная)
 - 7 - Поддерживаемые языки
 - 8 - Страна-производитель
 - 9 - Год релиза
 - 10 - Гарантийный срок
 - 11 - Рейтинг
 - 12 - Количество на складе
 - 13 - Описание
 - 0 - Прекратить редактирование
- Ваш выбор:1

Введите цену товара: 6000

+--+--+--+--+--Редактирование программного обеспечения (Nintendo The Legend of Zelda ID: SW3)--+--+--+--+--+

Рисунок 58 – Продолжение процесса редактирования товара

Id: SW3
Категория: Программное обеспечение
Бренд: Nintendo
Модель: The Legend of Zelda
Тип программного обеспечения: Игра
Срок лицензирования: 0 месяцев
Поддерживаемые операционные системы: Nintendo Switch
Системные требования: Nintendo Switch
Цифровая версия: Нет
Поддерживаемые языки: Русский, Английский
Цена: 6000 рублей
Страна-производитель: Япония
Год релиза: 2023
Гарантийный срок: 0 месяцев
Рейтинг: 4.76из 5!
Количество на складе: 3 шт.
Описание: популярная игра в жанре приключенческого файтинга

Выберите номер поля, которое собрались редактировать:
(Или введите 0, если вы уже отредактировали всё, что нужно):

- 1 - Цена
 - 2 - Тип программного обеспечения
 - 3 - Срок лицензии
 - 4 - Поддерживаемые операционные системы
 - 5 - Системные требования
 - 6 - Тип версии (цифровая/коробочная)
 - 7 - Поддерживаемые языки
 - 8 - Страна-производитель
 - 9 - Год релиза
 - 10 - Гарантийный срок
 - 11 - Рейтинг
 - 12 - Количество на складе
 - 13 - Описание
 - 0 - Прекратить редактирование
- Ваш выбор:8

Введите страну производителя: *Belgorod State University*

+--+--+--+--+--Редактирование программного обеспечения (Nintendo The Legend of Zelda ID: SW3)--+--+--+--+--+

Рисунок 59 – Продолжения процесса редактирования товара

Id: SW3
Категория: Программное обеспечение
Бренд: Nintendo
Модель: The Legend of Zelda
Тип программного обеспечения: Игра
Срок лицензирования: 0 месяцев
Поддерживаемые операционные системы: Nintendo Switch
Системные требования: Nintendo Switch
Цифровая версия: Нет
Поддерживаемые языки: Русский, Английский
Цена: 6000 рублей
Страна-производитель: Belgorod State University
Год релиза: 2023
Гарантийный срок: 0 месяцев
Рейтинг: 4.76из 5!
Количество на складе: 3 шт.
Описание: популярная игра в жанре приключенческого файтинга

Выберите номер поля, которое собрались редактировать:
(Или введите 0, если вы уже отредактировали всё, что нужно):

- 1 - Цена
 - 2 - Тип программного обеспечения
 - 3 - Срок лицензии
 - 4 - Поддерживаемые операционные системы
 - 5 - Системные требования
 - 6 - Тип версии (цифровая/коробочная)
 - 7 - Поддерживаемые языки
 - 8 - Страна-производитель
 - 9 - Год релиза
 - 10 - Гарантийный срок
 - 11 - Рейтинг
 - 12 - Количество на складе
 - 13 - Описание
 - 0 - Прекратить редактирование
- Ваш выбор: 0

===== Исходный вариант =====

Рисунок 60 – Продолжения процесса редактирования товара

Id: SW3
Категория: Программное обеспечение
Бренд: Nintendo
Модель: The Legend of Zelda
Тип программного обеспечения: Игра
Срок лицензирования: 0 месяцев
Поддерживаемые операционные системы: Nintendo Switch
Системные требования: Nintendo Switch
Цифровая версия: Нет
Поддерживаемые языки: Русский, Английский
Цена: 6100 рублей
Страна-производитель: Япония
Год релиза: 2023
Гарантийный срок: 0 месяцев
Рейтинг: 4.76из 5!
Количество на складе: 3 шт.
Описание: популярная игра в жанре приключенческого файтинга

===== Конечный вариант =====

Id: SW3
Категория: Программное обеспечение
Бренд: Nintendo
Модель: The Legend of Zelda
Тип программного обеспечения: Игра
Срок лицензирования: 0 месяцев
Поддерживаемые операционные системы: Nintendo Switch
Системные требования: Nintendo Switch
Цифровая версия: Нет
Поддерживаемые языки: Русский, Английский
Цена: 6000 рублей
Страна-производитель: Belgorod State University
Год релиза: 2023
Гарантийный срок: 0 месяцев
Рейтинг: 4.76из 5!
Количество на складе: 3 шт.
Описание: популярная игра в жанре приключенческого файтинга

Выберите одно из доступных действий:
1 - Сохранить все изменения
0 - Отменить изменения (оставить оригинал)
Ваш выбор:1

Ваши изменения были сохранены!

Редактирование (отмена редактирований) товара прошли успешно!

Рисунок 61 – Окончание процесса редактирования товара

На рисунке 62 показано содержимое консоли при удалении товара из каталога.

```
+--+--+--+--+--+Раздел продавца--+--+--+--+--+
Введите номер интересующего действия:
1 - Регистрация
2 - Авторизация
3 - Добавить товар
4 - Каталог товаров
5 - Поиск товара по id
6 - Изменить товар
7 - Удалить товар
0 - Выход в главное меню
Ваш выбор:7

+++++ Удаления товара +++++

Выберите одно из доступных действий:
1 - Точно удалить товар
0 - Вернуться назад
Ваш выбор:1

Введите id товара, который будет удалён:SW3

Вы точно хотите продолжить удаление товара?
ПОСЛЕДСТВИЯ БУДУТ НЕ ОБРАТИМЫ!
Введите 1 - если хотите продолжить процесс удаления товара!
Введите 0 - если хотите прервать процесс удаления!
Ваш выбор:0

Удаление было отменено!
```

Рисунок 62 – Процесс удаления товара из каталога

На этом возможности продавца как пользователя программы заканчиваются.

Таким образом конечная программа «Компьютерный магазин» реализует весь список требований, заложенный к ней на этапе проектирования. В процессе тестирования ошибок работы не обнаружено.

ЗАКЛЮЧЕНИЕ

В результате выполнения курсовой работы была успешно разработана программа «Компьютерный магазин», представляющая собой комплексное консольное приложение для автоматизации ключевых операций розничной торговли компьютерной техникой. В основу разработки положена объектно-ориентированная парадигма, что позволило создать систему, адекватно моделирующую предметную область через взаимодействие классов-сущностей.

Работа была выполнена в соответствии с поставленной целью и решением последовательных задач:

- В первом разделе на основе анализа специфики рынка компьютерной техники и эволюции бизнес-моделей (от офлайн-торговли к доминированию онлайн-каналов) были выявлены функциональные требования к системе, а также обоснован выбор языка Java и среды IntelliJ IDEA.

- Второй раздел посвящён проектированию архитектуры. Для визуализации сложной структуры будущей программы была разработана детализированная UML-диаграмма классов, отражающая многослойную архитектуру системы: слой модели данных (22 класса, включая иерархию товаров и пользователей), слой бизнес-логики (классы-менеджеры) и слой работы с данными. Диаграмма послужила точным техническим заданием для реализации.

- В третьем разделе выполнена непосредственная программная реализация на Java. Было создано работоспособное консольное приложение, предоставляющее функционал для двух ролей пользователей (клиент и продавец). Реализованы все заявленные требования: регистрация и авторизация, управление каталогом товаров со сложной фильтрацией и

сортировкой, механизм корзины и покупок, учёт остатков, а также сохранение состояния в файлы.

Итогом работы является полностью функционирующее приложение, лишённое недостатков ручного или частично автоматизированного учёта. Тестирование, продемонстрированное в работе, подтверждает корректность работы всех модулей и устойчивость к некорректным действиям пользователя. Полученная программа не только решает практическую задачу автоматизации, но и служит наглядным примером применения принципов ООП (наследование, инкапсуляция, полиморфизм).

Программа обладает потенциалом для дальнейшего развития: её можно расширить, добавив графический интерфейс (JavaFX), подключив СУБД для хранения данных или реализовав сетевой режим работы для многопользовательского доступа. Таким образом, все поставленные задачи решены, цель работы достигнута, а разработанное приложение представляет собой законченный, эффективный и масштабируемый программный продукт.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Бурлаков Алексей Сергеевич, Хмельнов Алексей Евгеньевич Язык спецификации архитектуры ЭВМ и периферийных устройств // Вестник БГУ. Математика, информатика. 2014. №9-3. URL: <https://cyberleninka.ru/article/n/yazyk-spetsifikatsii-arhitektury-evm-i-periferiynyh-ustroystv> (дата обращения: 05.01.2026).
2. Денисов И. А., Николаенко М. А., Сидоренко А. С., Гребенник О. Г., Игрунова С. В. СРАВНИТЕЛЬНЫЙ АНАЛИЗ ЯЗЫКОВ ПРОГРАММИРОВАНИЯ // Экономика и социум. 2015. №1-1 (14). URL: <https://cyberleninka.ru/article/n/sravnitelnyy-analiz-yazykov-programmirovaniya> (дата обращения: 05.01.2026).
3. Климова Н. В., Саутченкова Ю. Н. Развитие Интернет-торговли в России // Современные тенденции в экономике и управлении: новый взгляд. 2015. №37-1. URL: <https://cyberleninka.ru/article/n/razvitie-internet-torgovli-v-rossii> (дата обращения: 05.01.2026).
4. Краснокутский А. И. Рынок информационных компьютерных продуктов // Экономика и управление. 2007. №1. URL: <https://cyberleninka.ru/article/n/rynok-informatsionnyh-kompyuternyh-produktov> (дата обращения: 05.01.2026).
5. Макеева О. В., Сартаков М. В., Чернов Е. А. МОДЕЛИРОВАНИЕ ИНФОРМАЦИОННЫХ ПРОЦЕССОВ С ПОМОЩЬЮ UML // Инновации и инвестиции. 2021. №9. URL: <https://cyberleninka.ru/article/n/modelirovanie-informatsionnyh-protsessov-s-pomoschyu-uml> (дата обращения: 05.01.2026).
6. Маркелов, К. Д. Сравнительный анализ интегрированных сред разработки для языка Java / К. Д. Маркелов. — Текст : непосредственный // Молодой ученый. — 2021. — № 2 (344). — С. 26-27. — URL: <https://moluch.ru/archive/344/77367> (дата обращения: 05.01.2026).
7. Нестеренко Дмитрий Юрьевич Объектно-ориентированное программирование на примере языка Java // Научный журнал. 2016. №8 (9).

URL: <https://cyberleninka.ru/article/n/obektno-orientirovannoe-programmirovanie-na-primere-yazyka-java> (дата обращения: 05.01.2026).

8. Н. Х. Вафина Интернет-магазин // Мир новой экономики. 2010. №2 (8). URL: <https://cyberleninka.ru/article/n/internet-magazin> (дата обращения: 05.01.2026).

9. Объектно-ориентированное программирование (ООП): полное руководство для начинающих и профессионалов. — Текст : непосредственный // Молодой ученый. — 2017. — № 30 (164). — URL: <https://moluch.ru/archive/164/124493> (дата обращения: 05.01.2026).

10. Третьяченко А. Н. Общая характеристика российского рынка компьютерной техники // Сервис в России и за рубежом. 2007. №2. URL: <https://cyberleninka.ru/article/n/obschaya-harakteristika-rossiyskogo-rynka-kompyuternoy-tehniki> (дата обращения: 05.01.2026).

ПРИЛОЖЕНИЕ А

```
import java.util.*;
import java.util.stream.Collectors;

import static java.util.Comparator.comparing;

public class ComputerShop {
    private static final String address = "Belgorod, Pobedy street, 85b";
    private static final String name = "ENS";
    private static final String phone = "+79024292753";

    private static Client client = null;
    private static Seller seller = null;

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Migrator();// применяется при изменении методов классов

        boolean cycle = true;
        while (cycle)
        {
            System.out.println("\n\n+--+--+--+--+Компьютерный магазин--+--+--+--+");
            System.out.print("Введите номер интересующего раздела:\n" +
                "1 - Покупателю\n" +
                "2 - Продавцу\n" +
                "3 - Подробнее о магазине\n" +
                "0 - Выход\n" +
                "Ваш выбор:");
            int choice;

            try {
                choice = scanner.nextInt();
                switch (choice)
                {
                    case 1:
                        boolean cycleClient = true;
                        while (cycleClient)
                        {
                            System.out.println("\n\n+--+--+--+--+Раздел покупателя--+--+--+--+");
                            System.out.print("Введите номер интересующего действия:\n" +
                                "1 - Регистрация\n" +
                                "2 - Авторизация\n" +
                                "3 - Каталог товаров\n" +
                                "4 - Поиск товара по id\n" +
                                "5 - Проверить баланс\n" +
                                "6 - Корзина\n" +
                                "7 - Список покупок\n" +
                                "0 - Выход в главное меню\n" +
                                "Ваш выбор:");
                            int choiceClient;

                            try {
                                choiceClient = scanner.nextInt();
                                switch (choiceClient)
                                {
                                    case 1:
                                        AccountManager.registrationClient();
                                        break;
                                    case 2:
                                        client = AccountManager.authorizationClient();
                                        break;
                                    case 3:

                                        int printChoice;
                                        boolean print = true;
                                        while(print)
                                        {System.out.println("\n\n+--+--+--+--+Каталог товаров--+--+--+--+");
                                            System.out.print("\nВыберите интересующую категорию товара:\n" +
                                                "1 - Центральный процессор\n" +
                                                "2 - Материнская плата\n" +
                                                "3 - Видеокарта\n" +
                                                "4 - Оперативная память\n" +
                                                "5 - Постоянное запоминающее устройство\n" +
```

```

"6 - Блок питания\n" +
"7 - Корпус\n" +
"8 - Кулер\n" +
"9 - Программное обеспечение\n" +
"10 - Монитор\n" +
"11 - Мышь\n" +
"12 - Клавиатура\n" +
"0 - Выход в меню\n" +
"Ваш выбор:");

try {
    printChoice = scanner.nextInt();

    switch (printChoice)
    {
        case 1:
            handleProductMenu("Центральный процессор", scanner);
            break;
        case 2:
            handleProductMenu("Материнская плата", scanner);
            break;
        case 3:
            handleProductMenu("Видеокарта", scanner);
            break;
        case 4:
            handleProductMenu("Оперативная память", scanner);
            break;
        case 5:
            handleProductMenu("Постоянное запоминающее устройство", scanner);
            break;
        case 6:
            handleProductMenu("Блок питания", scanner);
            break;
        case 7:
            handleProductMenu("Корпус", scanner);
            break;
        case 8:
            handleProductMenu("Кулер", scanner);
            break;
        case 9:
            handleProductMenu("Программное обеспечение", scanner);
            break;
        case 10:
            handleProductMenu("Монитор", scanner);
            break;
        case 11:
            handleProductMenu("Мышь", scanner);
            break;
        case 12:
            handleProductMenu("Клавиатура", scanner);
            break;
        case 0:
            print = false;
            break;
        default:
            System.out.println("Такого варианта выбора нет!\n" +
                "Пожалуйста, введите корректное число!");
    }

} catch (InputMismatchException e) {
    System.out.println("Произошла ошибка!\n" +
        "Пожалуйста, введите корректное целочисленное значение выбранного варианта!\n" +
        "В прошлый раз вы ввели букву вместо числа!");
    scanner.nextLine();
} catch (Exception a) {
    System.out.println("Произошла неизвестная ошибка!");
    scanner.nextLine();
}

}

break;
case 4:
    //поиск товара
    int searchChoice;

```

```

boolean search = true;
while(search)
{
    System.out.println("\n\n+-+--+--+--+--Раздел поиска товара по id--+-+--+--+--+");
    System.out.print("\nВыберите одно из доступных действий:\n" +
        "1 - Выполнить поиск\n" +
        "0 - Выход в меню покупателя\n" +
        "Ваш выбор:");
    try {
        searchChoice = scanner.nextInt();

        switch (searchChoice)
        {
            case 1:
                productActionsMenu(scanner);
                break;
            case 0:
                search = false;
                break;
            default:
                System.out.println("Такого варианта выбора нет!\n" +
                    "Пожалуйста, введите корректное число!");
        }

    } catch (InputMismatchException e) {
        System.out.println("Произошла ошибка!\n" +
            "Пожалуйста, введите корректное целочисленное значение выбранного варианта!\n" +
            "В прошлый раз вы ввели букву вместо числа!");
        scanner.nextLine();
    }
    catch (Exception a) {
        System.out.println("Произошла неизвестная ошибка!");
        scanner.nextLine();
    }

}

break;
case 5:
    if(client != null)
    {
        // раздел баланса
        int balanceChoice;
        boolean balance = true;
        while(balance)
        {
            System.out.println("\n\n+-+--+--+--+--Баланс--+-+--+--+--+");
            System.out.println("Ваш баланс составляет: " + client.getBalance() + " рублей!");
            System.out.print("\nВыберите одно из доступных действий:\n" +
                "1 - Пополнить баланс\n" +
                "0 - Выход в меню покупателя\n" +
                "Ваш выбор:");
            try {
                balanceChoice = scanner.nextInt();

                switch (balanceChoice)
                {
                    {
                        case 1:
                            double sum;
                            boolean sumCycle = true;
                            while(sumCycle)
                            {
                                System.out.print("\nВыберите вносимую сумму в рублях:");

                                try {
                                    sum = scanner.nextDouble();
                                    client.addBalance(sum);
                                    FileManager.rewriteClient(client);

                                    sumCycle = false;
                                } catch (InputMismatchException e) {
                                    System.out.println("Произошла ошибка!\n" +
                                        "Пожалуйста, введите корректное целочисленное вносимой суммы!\n" +
                                        "В прошлый раз вы ввели букву вместо числа!");
                                    scanner.nextLine();
                                }
                            }
                        catch (Exception a) {
                            System.out.println("Произошла неизвестная ошибка!");
                        }
                    }
                }
            }
        }
    }
}

```

```

        scanner.nextLine();
    }
}
break;
case 0:
    balance = false;
    break;
default:
    System.out.println("Такого варианта выбора нет!\n" +
        "Пожалуйста, введите корректное число!");
}

} catch (InputMismatchException e) {
    System.out.println("Произошла ошибка!\n" +
        "Пожалуйста, введите корректное целочисленное значение выбранного варианта!\n" +
        "В прошлый раз вы ввели букву вместо числа!");
    scanner.nextLine();
}
catch (Exception a) {
    System.out.println("Произошла неизвестная ошибка!");
    scanner.nextLine();
}

}

}

else
{
    System.out.println("\n Для выполнения этого действия, сначала надо пройти процесс авторизации!");
}

break;

case 6:

    if(client != null)
    {
        // раздел корзины
        int basketChoice;
        boolean bascet = true;
        while(bascet)
        {
            System.out.println("\n\n+-+--+--+--+--+Ваша корзина--+-+--+--+--+");
            client.printShoppingCart();
            System.out.print("\nВыберите одно из доступных действий:\n" +
                "1 - Подробнее о каком-либо товаре\n" +
                "2 - Купить все товары\n" +
                "3 - Купить товар по его id\n" +
                "4 - Очистить корзину\n" +
                "5 - Удалить товар из корзины по его id\n" +
                "0 - Выход в меню покупателя\n" +
                "Ваш выбор:");
            try {
                basketChoice = scanner.nextInt();

                switch (basketChoice)
                {
                    case 1:
                        ProductManager.searchProductForClient();
                        int searchC;
                        boolean searchCCC = true;
                        while(searchCCC)
                        {
                            System.out.print("\nВведите 0 для возвращения к корзине:");
                            try {
                                searchC = scanner.nextInt();

                                switch (searchC)
                                {
                                    case 0:
                                        searchCCC = false;
                                        break;

```

```

        default:
            System.out.println("Такого варианта выбора нет!\n" +
                "Пожалуйста, введите корректное число!");
        }

    } catch (InputMismatchException e) {
        System.out.println("Произошла ошибка!\n" +
            "Пожалуйста, введите корректное целочисленное значение выбранного варианта!\n" +
            "В прошлый раз вы ввели букву вместо числа!");
        scanner.nextLine();
    }
    catch (Exception a) {
        System.out.println("Произошла неизвестная ошибка!");
        scanner.nextLine();
    }
}
break;
case 2:
    int buyALLChoice;
    boolean buyALLCycle = true;
    while(buyALLCycle)
    {

        System.out.print("\nВы точно уверены в том, что хотите купить все товары из вашей корзины?\n" +
            "Выберите одно из доступных действий:\n" +
            "1 - Подтвердить покупку всех товаров\n" +
            "0 - Отменить процесс покупки всех товаров\n" +
            "Ваш выбор:");

        try {
            buyALLChoice = scanner.nextInt();
            scanner.nextLine();

            switch (buyALLChoice)
            {
                case 1:
                    client.buyALLShoppingCard(scanner);
                    FileManager.rewriteClient(client);
                    buyALLCycle = false;
                    break;

                case 0:
                    buyALLCycle = false;
                    System.out.println("\nПроцесс покупки всех товаров был отменён!");
                    break;

                default:
                    System.out.println("Такого варианта выбора нет!\n" +
                        "Пожалуйста, введите корректное число!");
            }
        }

        catch (InputMismatchException e) {
            System.out.println("Произошла ошибка!\n" +
                "Пожалуйста, введите корректное целочисленное значение выбранного варианта!\n" +
                "В прошлый раз вы ввели букву вместо числа!");
            scanner.nextLine();
        }
        catch (Exception a) {
            System.out.println("Произошла неизвестная ошибка!");
            scanner.nextLine();
        }
    }

}
break;
case 3:
    int byeToIDChoice;
    boolean byeToIDCycle = true;
    while(byeToIDCycle)
    {

        System.out.print("\nВы точно уверены в том, что хотите купить товар?\n" +
            "Выберите одно из доступных действий:\n" +
            "1 - Подтвердить покупку товара\n" +
            "0 - Отменить процесс покупки товара\n" +
            "Ваш выбор:");

        try {
            byeToIDChoice = scanner.nextInt();
            scanner.nextLine();

```



```

switch (byeToIDChoice)
{
    case 1:
        System.out.print("\nВведите id товара, который хотите купить:");
        String idToBuy = scanner.nextLine();
        client.buyProductInShoppingCartById(idToBuy, scanner);
        FileManager.rewriteClient(client);
        byeToIDCycle = false;
        break;

    case 0:
        byeToIDCycle = false;
        System.out.println("\nПроцесс покупки товара был отменён!");
        break;

    default:
        System.out.println("Такого варианта выбора нет!\n" +
            "Пожалуйста, введите корректное число!");
}

} catch (InputMismatchException e) {
    System.out.println("Произошла ошибка!\n" +
        "Пожалуйста, введите корректное целочисленное значение выбранного варианта!\n" +
        "В прошлый раз вы ввели букву вместо числа!");
    scanner.nextLine();
}
catch (Exception a) {
    System.out.println("Произошла неизвестная ошибка!");
    scanner.nextLine();
}

}

break;
case 4:
    int clearChoice;
    boolean clearCycle = true;
    while(clearCycle)
    {

        System.out.print("\nВы точно уверены в том, что хотите удалить всё содержимое вашей корзины?\n"

            "Выберите одно из доступных действий:\n" +
            "1 - Подтвердить очистку корзины\n" +
            "0 - Отменить процесс очистки корзины\n" +
            "Ваш выбор:");

        try {
            clearChoice = scanner.nextInt();
            scanner.nextLine();

            switch (clearChoice)
            {
                case 1:
                    client.clearShoppingCart();
                    clearCycle = false;
                    FileManager.rewriteClient(client);
                    break;

                case 0:
                    clearCycle = false;
                    System.out.println("\nПроцесс очистки корзины был отменён!");
                    break;

                default:
                    System.out.println("Такого варианта выбора нет!\n" +
                        "Пожалуйста, введите корректное число!");
            }

        } catch (InputMismatchException e) {
            System.out.println("Произошла ошибка!\n" +
                "Пожалуйста, введите корректное целочисленное значение выбранного варианта!\n" +
                "В прошлый раз вы ввели букву вместо числа!");
            scanner.nextLine();
        }
        catch (Exception a) {
            System.out.println("Произошла неизвестная ошибка!");
            scanner.nextLine();
        }
    }
}

```

```

    }
    break;
case 5:
    int removeToIDChoice;
    boolean removeToIDCycle = true;
    while(removeToIDCycle)
    {

        System.out.print("\nВы точно уверены в том, что хотите удалить товар из вашей корзины?\n" +
            "Выберите одно из доступных действий:\n" +
            "1 - Подтвердить удаление товара из корзины\n" +
            "0 - Отменить процесс удаления товара из корзины\n" +
            "Ваш выбор:");
        try {
            removeToIDChoice = scanner.nextInt();
            scanner.nextLine();

            switch (removeToIDChoice)
            {
                case 1:
                    System.out.print("\nВведите id товара, который хотите удалить из корзины:");
                    String idToRemove = scanner.nextLine();
                    client.removeProductInShoppingCartById(idToRemove);
                    FileManager.rewriteClient(client);
                    removeToIDCycle = false;
                    break;

                case 0:
                    removeToIDCycle = false;
                    System.out.println("\nПроцесс удаления товара из корзины был отменён!");
                    break;
                default:
                    System.out.println("Такого варианта выбора нет!\n" +
                        "Пожалуйста, введите корректное число!");
            }
        } catch (InputMismatchException e) {
            System.out.println("Произошла ошибка!\n" +
                "Пожалуйста, введите корректное целочисленное значение выбранного варианта!\n" +
                "В прошлый раз вы ввели букву вместо числа!");
            scanner.nextLine();
        }
        catch (Exception a) {
            System.out.println("Произошла неизвестная ошибка!");
            scanner.nextLine();
        }
    }
    break;
case 0:
    bascet = false;
    break;
default:
    System.out.println("Такого варианта выбора нет!\n" +
        "Пожалуйста, введите корректное число!");
}

} catch (InputMismatchException e) {
    System.out.println("Произошла ошибка!\n" +
        "Пожалуйста, введите корректное целочисленное значение выбранного варианта!\n" +
        "В прошлый раз вы ввели букву вместо числа!");
    scanner.nextLine();
}
catch (Exception a) {
    System.out.println("Произошла неизвестная ошибка!");
    scanner.nextLine();
}

}

}

else
{
    System.out.println("\n Для выполнения этого действия, сначала надо пройти процесс авторизации!");
}
}

```

```

        break;
    case 7:

        if(client != null)
        {
            int shoppingListChoice;
            boolean shoppingList = true;
            while(shoppingList)
            {
                System.out.println("\n\n+--+--+--+--+Ваш список покупок--+--+--+--+");
                client.printShoppingList();
                System.out.print("\nВыберите одно из доступных действий:\n" +
                    "1 - Подробнее о товаре товаре:\n" +
                    "0 - Выход в меню покупателя\n" +
                    "Ваш выбор:");
                try {
                    shoppingListChoice = scanner.nextInt();

                    switch (shoppingListChoice)
                    {
                        case 1:
                            ProductManager.searchProductForClient();
                            int searchC;
                            boolean searchCCC = true;
                            while(searchCCC)
                            {

                                System.out.print("\nВведите 0 для возвращения к списку покупок:");
                                try {
                                    searchC = scanner.nextInt();

                                    switch (searchC)
                                    {
                                        case 0:
                                            searchCCC = false;
                                            break;
                                        default:
                                            System.out.println("Такого варианта выбора нет!\n" +
                                                "Пожалуйста, введите корректное число!");
                                    }
                                }

                                } catch (InputMismatchException e) {
                                    System.out.println("Произошла ошибка!\n" +
                                        "Пожалуйста, введите корректное целочисленное значение выбранного варианта!\n" +
                                        "В прошлый раз вы ввели букву вместо числа!");
                                    scanner.nextLine();
                                }
                                catch (Exception a) {
                                    System.out.println("Произошла неизвестная ошибка!");
                                    scanner.nextLine();
                                }
                            }

                            break;
                        case 0:
                            shoppingList = false;
                            break;
                        default:
                            System.out.println("Такого варианта выбора нет!\n" +
                                "Пожалуйста, введите корректное число!");
                    }

                } catch (InputMismatchException e) {
                    System.out.println("Произошла ошибка!\n" +
                        "Пожалуйста, введите корректное целочисленное значение выбранного варианта!\n" +
                        "В прошлый раз вы ввели букву вместо числа!");
                    scanner.nextLine();
                }
                catch (Exception a) {
                    System.out.println("Произошла неизвестная ошибка!");
                    scanner.nextLine();
                }
            }
        }
    }
}

```

```

    }

    else
    {
        System.out.println("\n Для выполнения этого действия, сначала надо пройти процесс авторизации!");
    }

    break;
case 0:
    cycleClient = false;
    break;
default:
    System.out.println("Такого варианта выбора нет!\n" +
        "Пожалуйста, введите корректную цифру!");
}

} catch (InputMismatchException e) {
    System.out.println("Произошла ошибка!\n" +
        "Пожалуйста, введите корректное целочисленное значение выбранного варианта!\n" +
        "В прошлый раз вы ввели букву вместо числа!");
    scanner.nextLine();
}
catch (Exception a) {
    System.out.println("Произошла неизвестная ошибка!");
    scanner.nextLine();
}

}

break;
case 2:
boolean cycleSeller = true;
while (cycleSeller)
{
    System.out.println("\n\n+--+--+--+--+Раздел продавца--+--+--+--+");
    System.out.print("Введите номер интересующего действия:\n" +
        "1 - Регистрация\n" +
        "2 - Авторизация\n" +
        "3 - Добавить товар\n" +
        "4 - Каталог товаров\n" +
        "5 - Поиск товара по id\n" +
        "6 - Изменить товар\n" +
        "7 - Удалить товар\n" +
        "0 - Выход в главное меню\n" +
        "Ваш выбор:");
    int choiceSeller;

    try {
        choiceSeller = scanner.nextInt();
        switch (choiceSeller)
        {
            case 1://Регистрация
                AccountManager.registrationSeller();
                break;
            case 2://Авторизация
                seller = AccountManager.authorizationSeller();
                break;
            case 3://Добавление товара
                if(seller!=null)
                {
                    seller.registrationProduct();
                }

                else
                {
                    System.out.println("\nПродавец не авторизован!");
                }
                break;
            case 4://Каталог товаров
                if(seller!=null)
                {
                    int printChoice;
                    boolean print = true;
                    while(print)
                    {System.out.println("\n\n+--+--+--+--+Каталог товаров--+--+--+--+");
                        System.out.print("\nВыберите интересующую категорию товара:\n" +

```

```

"1 - Центральный процессор\n" +
"2 - Материнская плата\n" +
"3 - Видеокарта\n" +
"4 - Оперативная память\n" +
"5 - Постоянное запоминающее устройство\n" +
"6 - Блок питания\n" +
"7 - Корпус\n" +
"8 - Кулер\n" +
"9 - Программное обеспечение\n" +
"10 - Монитор\n" +
"11 - Мышь\n" +
"12 - Клавиатура\n" +
"0 - Выход в меню\n" +
"Ваш выбор:");

```

```

try {
    printChoice = scanner.nextInt();

    switch (printChoice)
    {
        case 1:
            ProductManager.printFullInformationCPU();
            handleProductSellerMenu(scanner);
            break;
        case 2:
            ProductManager.printFullInformationMotherboard();
            handleProductSellerMenu(scanner);
            break;
        case 3:
            ProductManager.printFullInformationGPU();
            handleProductSellerMenu(scanner);
            break;
        case 4:
            ProductManager.printFullInformationRAM();
            handleProductSellerMenu(scanner);
            break;
        case 5:
            ProductManager.printFullInformationStorage();
            handleProductSellerMenu(scanner);
            break;
        case 6:
            ProductManager.printFullInformationPowerSupply();
            handleProductSellerMenu(scanner);
            break;
        case 7:
            ProductManager.printFullInformationCase();
            handleProductSellerMenu(scanner);
            break;
        case 8:
            ProductManager.printFullInformationCooler();
            handleProductSellerMenu(scanner);
            break;
        case 9:
            ProductManager.printFullInformationSoftware();
            handleProductSellerMenu(scanner);
            break;
        case 10:
            ProductManager.printFullInformationMonitor();
            handleProductSellerMenu(scanner);
            break;
        case 11:
            ProductManager.printFullInformationMouse();
            handleProductSellerMenu(scanner);
            break;
        case 12:
            ProductManager.printFullInformationKeyboard();
            handleProductSellerMenu(scanner);
            break;
        case 0:
            print = false;
            break;
        default:
            System.out.println("Такого варианта выбора нет!\n" +
                "Пожалуйста, введите корректное число!");
    }
} catch (InputMismatchException e) {

```

```

        System.out.println("Произошла ошибка!\n" +
            "Пожалуйста, введите корректное целочисленное значение выбранного варианта!\n" +
            "В прошлый раз вы ввели букву вместо числа!");
        scanner.nextLine();
    }
    catch (Exception a) {
        System.out.println("Произошла неизвестная ошибка!");
        scanner.nextLine();
    }
}

else
{
    System.out.println("\nПродавец не авторизован!");
}

break;
case 5://Поиск по id
if(seller!=null)
{
    int searchC;
    boolean searchCCC = true;
    while(searchCCC)
    {
        System.out.println("==== Поиск товара по ID =====");
        ProductManager.searchProductForSeller();

        System.out.print("\nВведите 0 для возвращения к меню продавца:\n" +
            "Введите 1 - чтобы повторно выполнить поиск:\n" +
            "Ваш выбор:");
        try {
            searchC = scanner.nextInt();

            switch (searchC)
            {
                case 0:
                    searchCCC = false;
                    break;
                case 1:

                    break;
                default:
                    System.out.println("Такого варианта выбора нет!\n" +
                        "Пожалуйста, введите корректное число!");
            }
        } catch (InputMismatchException e) {
            System.out.println("Произошла ошибка!\n" +
                "Пожалуйста, введите корректное целочисленное значение выбранного варианта!\n" +
                "В прошлый раз вы ввели букву вместо числа!");
            scanner.nextLine();
        }
        catch (Exception a) {
            System.out.println("Произошла неизвестная ошибка!");
            scanner.nextLine();
        }
    }
}

else
{
    System.out.println("\nПродавец не авторизован!");
}

break;
case 6://Изменить товар
if(seller!=null)
{
    ProductManager.reWriteTovarAndProduct(scanner);
}

else

```

```

        {
            System.out.println("\nПродавец не авторизован!");
        }

        break;
    case 7://Удалить товар
        if(seller!=null)
        {
            ProductManager.removeTovarAndProduct(scanner);
        }

        else
        {
            System.out.println("\nПродавец не авторизован!");
        }

        break;
    case 0:
        cycleSeller = false;
        break;
    default:
        System.out.println("Такого варианта выбора нет!\n" +
            "Пожалуйста, введите корректную цифру!");
    }

} catch (InputMismatchException e) {
    System.out.println("Произошла ошибка!\n" +
        "Пожалуйста, введите корректное целочисленное значение выбранного варианта!\n" +
        "В прошлый раз вы ввели букву вместо числа!");
    scanner.nextLine();
}
catch (Exception a) {
    System.out.println("Произошла неизвестная ошибка!");
    scanner.nextLine();
}

}
break;
case 3:
    aboutShop();
    break;
case 0:
    System.out.println("Выражаю благодарность за пользование программой!\n" +
        "До новых встреч!");
    cycle = false;
    break;

default:
    System.out.println("Такого варианта выбора нет!\n" +
        "Пожалуйста, введите корректную цифру!");
}

} catch (InputMismatchException e) {
    System.out.println("Произошла ошибка!\n" +
        "Пожалуйста, введите корректное целочисленное значение выбранного варианта!\n" +
        "В прошлый раз вы ввели букву вместо числа!");
    scanner.nextLine();
}
catch (Exception a) {
    System.out.println("Произошла неизвестная ошибка!");
    scanner.nextLine();
}

}

scanner.close();

}

public static void aboutShop(){
    System.out.println("\nНазвание магазина:" + name);
    System.out.println("Адрес:" + address);
    System.out.println("Контактный телефон:" + phone);
    System.out.println("\n\nДанный компьютерный магазин не существует в реальности,\n" +
        "а представляет собой лишь некоммерческую программу,\n" +
        "написанную студентом в качестве курсовой работы,\n" +
        "все совпадения с возможной логикой работы магазина 'DNS' вовсе не случайны,\n" +
        "так как лучший способ создать что-то хорошее - брать пример с лучших!");
}

```

```

    }

    public static String getName()
    {
        return name;
    }

    private static void handleProductMenu(String categoryName, Scanner scanner) {

        Map<String, Product> products_0 = ProductManager.printMiniInfo(categoryName);

        List<Product> products = new ArrayList<>(products_0.values());
        List<Product> products_no_izmen = new ArrayList<>(products_0.values());

        int productChoice;
        boolean productCycle = true;

        while(productCycle) {
            System.out.print("\nВыберите одно из доступных действий:\n" +
                "1 - Подробнее о товаре\n" +
                "2 - Вывести товары только в выделенном диапазоне\n" +
                "3 - Вывести товары только ниже введенной цены\n" +
                "4 - Вывести товары только выше указанной цены\n" +
                "5 - Сортировать по увеличению цены\n" +
                "6 - Сортировать по уменьшению цены\n" +
                "7 - Сортировать по уменьшению рейтинга\n" +
                "8 - Сортировать по увеличению рейтинга\n" +
                "9 - Сбросить сортировки\n" +
                "0 - Вернуться к каталогу товаров\n" +
                "Ваш выбор: ");

            try {
                productChoice = scanner.nextInt();

                switch (productChoice) {
                    case 1:
                        productActionsMenu(scanner);
                        break;
                    case 2:

                        double minPrice=0;
                        double maxPrice=0;
                        boolean priceMinMax = true;
                        while (priceMinMax){

                            System.out.print("\nВведите нижний порог интересующей цены:");
                            try{
                                minPrice = scanner.nextDouble();

                                try {
                                    System.out.print("\nВведите верхний порог интересующей цены:");
                                    maxPrice = scanner.nextDouble();
                                    if(maxPrice<minPrice){
                                        System.out.println("\nОшибка! Верхний порог диапазона не может быть меньше нижнего!");
                                        continue;
                                    }
                                }

                                double finalMinPrice = minPrice;
                                double finalMaxPrice = maxPrice;
                                products = products_no_izmen.stream().filter(p -> p.getPrice() >= finalMinPrice).
                                    filter(p -> p.getPrice() <= finalMaxPrice).filter(p -
>p.getCategory().equals(categoryName)).collect(Collectors.toList());

                                if(products.isEmpty()){
                                    System.out.println("\nНе найдено товаров, удовлетворяющих критериям вашего отбора!");
                                } else {
                                    System.out.printf("\n=====Товары в ценовом диапазоне от %.2f до %.2f
рублей=====
", minPrice, maxPrice);
                                    for(Product product : products){
                                        product.getMiniInfo();
                                        System.out.println("=====");
                                    }
                                }
                            }
                        }
                    }
                }
            }
        }
    }

```



```

        priceMinMax = false;

        } catch (InputMismatchException e) {
            System.out.println("Произошла ошибка!\n" +
                "Пожалуйста, введите корректное целочисленное значение выбранного варианта!\n" +
                "В прошлый раз вы ввели букву вместо числа!");
            scanner.nextLine();
        } catch (Exception a) {
            System.out.println("Произошла неизвестная ошибка!");
            scanner.nextLine();
        }
    }

    } catch (InputMismatchException e) {
        System.out.println("Произошла ошибка!\n" +
            "Пожалуйста, введите корректное целочисленное значение выбранного варианта!\n" +
            "В прошлый раз вы ввели букву вместо числа!");
        scanner.nextLine();
    } catch (Exception a) {
        System.out.println("Произошла неизвестная ошибка!");
        scanner.nextLine();
    }
}

break;
case 3:

    double maxP = 0;
    boolean maxip = true;
    while(maxip){
        System.out.print("\nВведите максимальную допустимую цену:");
        try{

            maxP = scanner.nextDouble();

            double finalMaxP = maxP;
            products = products_no_izmen.stream().filter(p -> p.getPrice() <= finalMaxP).
                filter(p -> p.getCategory().equals(categoryName)).collect(Collectors.toList());

            if(products.isEmpty()){
                System.out.println("\nНе найдено товаров, удовлетворяющих критериям вашего отбора!");
            } else {
                System.out.printf("\n=====Товары, стоящие меньше %.2f рублей=====\\n",
maxP);

                for(Product product : products){
                    product.getMiniInfo();
                    System.out.println("=====");
                }
            }
            maxip = false;

        } catch (InputMismatchException e) {
            System.out.println("Произошла ошибка!\n" +
                "Пожалуйста, введите корректное целочисленное значение выбранного варианта!\n" +
                "В прошлый раз вы ввели букву вместо числа!");
            scanner.nextLine();
        } catch (Exception a) {
            System.out.println("Произошла неизвестная ошибка!");
            scanner.nextLine();
        }
    }

break;
case 4:
    double minP = 0;
    boolean minip = true;
    while(minip){
        System.out.print("\nВведите минимальную допустимую цену:");
        try{

            minP = scanner.nextDouble();
            double finalMinP = minP;

            products = products_no_izmen.stream().filter(p-> p.getPrice() >= finalMinP).
                filter(p -> p.getCategory().equals(categoryName)).collect(Collectors.toList());

```

```

        if(products.isEmpty()){
            System.out.println("\nНе найдено товаров, удовлетворяющих критериям вашего отбора!");
        } else {
            System.out.printf("\n=====Товары, стоящие больше %.2f рублей=====\\n", minP);
            for(Product product : products){
                product.getMiniInfo();
                System.out.println("=====");
            }
        }
        minip = false;

    } catch (InputMismatchException e) {
        System.out.println("Произошла ошибка!\\n" +
            "Пожалуйста, введите корректное целочисленное значение выбранного варианта!\\n" +
            "В прошлый раз вы ввели букву вместо числа!");
        scanner.nextLine();
    } catch (Exception a) {
        System.out.println("Произошла неизвестная ошибка!");
        scanner.nextLine();
    }
}

break;
case 5:
    products = products_no_izmen.stream().filter(p -> p.getCategory().equals(categoryName))
        .collect(Collectors.toList());
    products.sort(comparing(Product::getPrice));
    if(products.isEmpty()){
        System.out.println("\nНе найдено товаров, удовлетворяющих критериям вашего отбора!");
    } else {
        System.out.print("\n=====Товары, отсортированные по увеличению цены=====\\n");
        for(Product product : products){
            product.getMiniInfo();
            System.out.println("=====");
        }
    }

    break;
case 6:
    products = products_no_izmen.stream().filter(p -> p.getCategory().equals(categoryName))
        .collect(Collectors.toList());
    products.sort(comparing(Product::getPrice).reversed());
    if(products.isEmpty()){
        System.out.println("\nНе найдено товаров, удовлетворяющих критериям вашего отбора!");
    } else {
        System.out.print("\n=====Товары, отсортированные по уменьшению цены=====\\n");
        for(Product product : products){
            product.getMiniInfo();
            System.out.println("=====");
        }
    }

    break;
case 7:
    products = products_no_izmen.stream().filter(p -> p.getCategory().equals(categoryName))
        .collect(Collectors.toList());
    products.sort(comparing(Product::getRating).reversed());
    if(products.isEmpty()){
        System.out.println("\nНе найдено товаров, удовлетворяющих критериям вашего отбора!");
    } else {
        System.out.print("\n=====Товары, отсортированные по уменьшению
рейтинга=====\\n");
        for(Product product : products){
            product.getMiniInfo();
            System.out.println("=====");
        }
    }

    break;
case 8:
    products = products_no_izmen.stream().filter(p -> p.getCategory().equals(categoryName))
        .collect(Collectors.toList());
    products.sort(comparing(Product::getRating));
    if(products.isEmpty()){
        System.out.println("\nНе найдено товаров, удовлетворяющих критериям вашего отбора!");
    } else {
        System.out.print("\n=====Товары, отсортированные по увеличению
рейтинга=====\\n");

```

```

        for(Product product : products){
            product.getMiniInfo();
            System.out.println("=====");
        }
    }

    break;
case 9:
    ProductManager.printMiniInfo(categoryName);
    products = products_no_izmen;
    break;
case 0:
    productCycle = false;
    break;
default:
    System.out.println("Такого варианта выбора нет!\n" +
        "Пожалуйста, введите корректное число!");
}

} catch (InputMismatchException e) {
    System.out.println("Произошла ошибка!\n" +
        "Пожалуйста, введите корректное целочисленное значение выбранного варианта!\n" +
        "В прошлый раз вы ввели букву вместо числа!");
    scanner.nextLine();
} catch (Exception a) {
    System.out.println("Произошла неизвестная ошибка!");
    scanner.nextLine();
}
}
}

private static void productActionsMenu( Scanner scanner) {
    Product product = ProductManager.searchProductForClient();
    int choice;
    boolean cycle = true;

    while(cycle) {
        System.out.print("\nВыберите одно из доступных действий:\n" +
            "1 - Купить товар\n" +
            "2 - Положить товар в корзину\n" +
            "3 - Авторизоваться\n" +
            "0 - Назад\n" +
            "Ваш выбор: ");

        try {
            choice = scanner.nextInt();

            switch (choice) {
                case 1:
                    if(client != null) {
                        ProductManager.buyProductTovar(product,client,scanner);
                    } else {
                        System.out.println("\nДля выполнения этого действия сначала надо пройти процесс авторизации!");
                    }
                    break;

                case 2:
                    if(client != null) {
                        client.addShoppingCart(product);
                        FileManager.rewriteClient(client);
                    } else {
                        System.out.println("\nДля выполнения этого действия сначала надо пройти процесс авторизации!");
                    }
                    break;

                case 3:
                    client = AccountManager.authorizationClient();
                    break;

                case 0:
                    cycle = false;
                    break;

                default:
                    System.out.println("Такого варианта выбора нет!\n" +
                        "Пожалуйста, введите корректное число!");
            }
        }
    }
}

```

```

    }

    } catch (InputMismatchException e) {
        System.out.println("Произошла ошибка!\n" +
            "Пожалуйста, введите корректное целочисленное значение выбранного варианта!\n" +
            "В прошлый раз вы ввели букву вместо числа!");
        scanner.nextLine();
    } catch (Exception a) {
        System.out.println("Произошла неизвестная ошибка!");
        scanner.nextLine();
    }
}

}

}

public static void handleProductSellerMenu( Scanner scanner){

    int printChoice;
    boolean print = true;
    while(print)
    {
        System.out.print("\nВыберите одно из доступных действий:\n" +
            "1 - Добавить товар\n" +
            "2 - Изменить товар\n" +
            "3 - Удалить товар\n" +
            "0 - Вернуться к каталогу товаров\n" +
            "Ваш выбор:");

        try {
            printChoice = scanner.nextInt();

            switch (printChoice)
            {
                case 1:
                    seller.registrationProduct();
                    break;
                case 2:
                    ProductManager.reWriteTovarAndProduct(scanner);
                    break;
                case 3:
                    ProductManager.removeTovarAndProduct(scanner);
                    break;
                case 0:
                    print = false;
                    break;
                default:
                    System.out.println("Такого варианта выбора нет!\n" +
                        "Пожалуйста, введите корректное число!");
            }
        } catch (InputMismatchException e) {
            System.out.println("Произошла ошибка!\n" +
                "Пожалуйста, введите корректное целочисленное значение выбранного варианта!\n" +
                "В прошлый раз вы ввели букву вместо числа!");
            scanner.nextLine();
        }
        catch (Exception a) {
            System.out.println("Произошла неизвестная ошибка!");
            scanner.nextLine();
        }
    }

}

}

public static void Migrator(){
    System.out.println("=== Начало миграции товаров ===");

    // Просто загружаем и сразу сохраняем каждый тип товаров
    // Это перезапишет объекты как "новые"

    // 1. Процессоры
    Map<String, CPU> cpuMap = FileManager.loadCPU();
    FileManager.saveCPU(cpuMap);
    System.out.println("CPU мигрированы: " + cpuMap.size());
}

```

```

// 2. RAM
Map<String, RAM> ramMap = FileManager.loadRAM();
FileManager.saveRAM(ramMap);
System.out.println("RAM мигрированы: " + ramMap.size());

// 3. Материнские платы
Map<String, MotherBoard> mbMap = FileManager.loadMotherBoard();
FileManager.saveMotherBoard(mbMap);
System.out.println("MotherBoard мигрированы: " + mbMap.size());

// 4. Видеокарты
Map<String, GPU> gpuMap = FileManager.loadGPU();
FileManager.saveGPU(gpuMap);
System.out.println("GPU мигрированы: " + gpuMap.size());

// 5. Накопители
Map<String, Storage> storageMap = FileManager.loadStorage();
FileManager.saveStorage(storageMap);
System.out.println("Storage мигрированы: " + storageMap.size());

// 6. Блоки питания
Map<String, PowerSupply> psMap = FileManager.loadPowerSupply();
FileManager.savePowerSupply(psMap);
System.out.println("PowerSupply мигрированы: " + psMap.size());

// 7. Корпуса
Map<String, Case> caseMap = FileManager.loadCase();
FileManager.saveCase(caseMap);
System.out.println("Case мигрированы: " + caseMap.size());

// 8. Кулеры
Map<String, Cooler> coolerMap = FileManager.loadCooler();
FileManager.saveCooler(coolerMap);
System.out.println("Cooler мигрированы: " + coolerMap.size());

// 9. ПО
Map<String, Software> softwareMap = FileManager.loadSoftware();
FileManager.saveSoftware(softwareMap);
System.out.println("Software мигрированы: " + softwareMap.size());

// 10. Мониторы
Map<String, Monitor> monitorMap = FileManager.loadMonitor();
FileManager.saveMonitor(monitorMap);
System.out.println("Monitor мигрированы: " + monitorMap.size());

// 11. Мыши
Map<String, Mouse> mouseMap = FileManager.loadMouse();
FileManager.saveMouse(mouseMap);
System.out.println("Mouse мигрированы: " + mouseMap.size());

// 12. Клавиатуры
Map<String, Keyboard> keyboardMap = FileManager.loadKeyboard();
FileManager.saveKeyboard(keyboardMap);
System.out.println("Keyboard мигрированы: " + keyboardMap.size());

// 13. Общая карта продуктов
Map<String, Product> productMap = FileManager.loadProduct0();
FileManager.saveProduct0(productMap);
System.out.println("Общая карта продуктов мигрирована: " + productMap.size());

System.out.println("=== Миграция завершена ===");
}
}

```

Рисунок 1 – Код класса ComputerShop

https://github.com/rtn12/Kursovaya_Rabota.git

git@github.com:rtn12/Kursovaya_Rabota.git

gh repo clone rtn12/Kursovaya_Rabota

Рисунок 2 – Ссылки на репозиторий «GitHub» для копирования проекта

